Classification of Forest Cover Type

Training and Testing using KNIME Platform

Overview

This document presents the complete workflow for classifying forest cover types using five supervised machine learning algorithms: Naïve Bayes, Decision Tree, K-Nearest Neighbors (KNN), Support Vector Machine (SVM), and Neural Network. The objective is to document the implementation process, from initial dataset preparation to the development of individual classification workflows.

The process begins with loading the Forest Cover Type dataset using the CSV Reader node, followed by the necessary data preprocessing steps. The resulting preprocessed dataset is exported using the CSV Writer node and used as the input for the subsequent classification workflows. Each algorithm is implemented in a separate KNIME workflow file, allowing its configuration and execution process to be examined independently.

For each workflow, the relevant nodes, their configurations, and their outputs are documented. Explanations are provided for important settings and processing steps to clarify the purpose of each node and its contribution to the classification process. This documentation serves as a technical reference for understanding how the individual models were constructed and executed in KNIME, supporting the accompanying analysis and results report, which presents the comparative evaluation of the models and the rationale for selecting the most suitable classifier.

Table of Contents

1. Preparing the Dataset 3
    1.1 Importing Data to the Workflow 3
    1.2 Creating a sample 3
2. Data Preprocessing 4
    2.1 Detect and handle missing values 4
    2.2 Detect and handle outliers 5
        2.2.1 Why Missing Values Appear After the Numeric Outliers Node 6
    2.3 Data transformation: Normalization 7
    2.4 Feature selection and engineering 8
3. Model Training and Testing 13
    (a) Naive Bayes 13
    (b) Decision Tree 19
    (c) K-Nearest Neighbors (KNN) 23
    (d) Support Vector Machine (SVM) 27
    (e) Neural Network 31

1. Preparing the Dataset

1.1 Importing Data to the Workflow

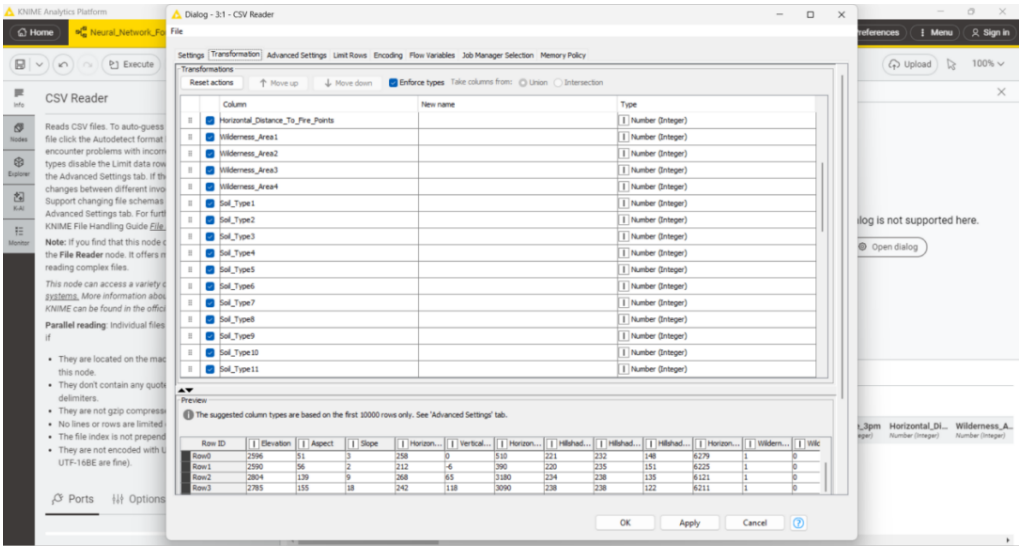


Figure 1: CSV Reader Node Configuration

By default, the type of the target variable *Cover_Type* was Number. For classification, it is essential for the target variable to be a class type or a category. Therefore, the data type of *Cover_Type* was changed from *Number* to String.

1.2 Creating a sample

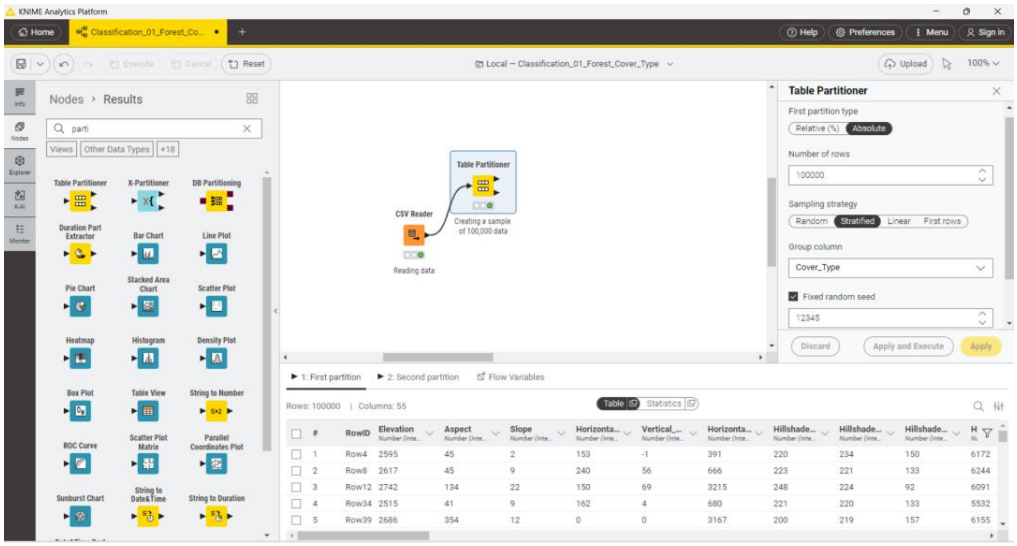


Figure 2: Table Partitioner Node

A **stratified sampling** method was used to select exactly 100,000 records while keeping the same class balance of the *Cover_Type* variable. The **absolute partition** size ensured an exact sample count, and a **fixed random seed** (12345) was used for reproducibility

2. Data Preprocessing

2.1 Detect and handle missing values

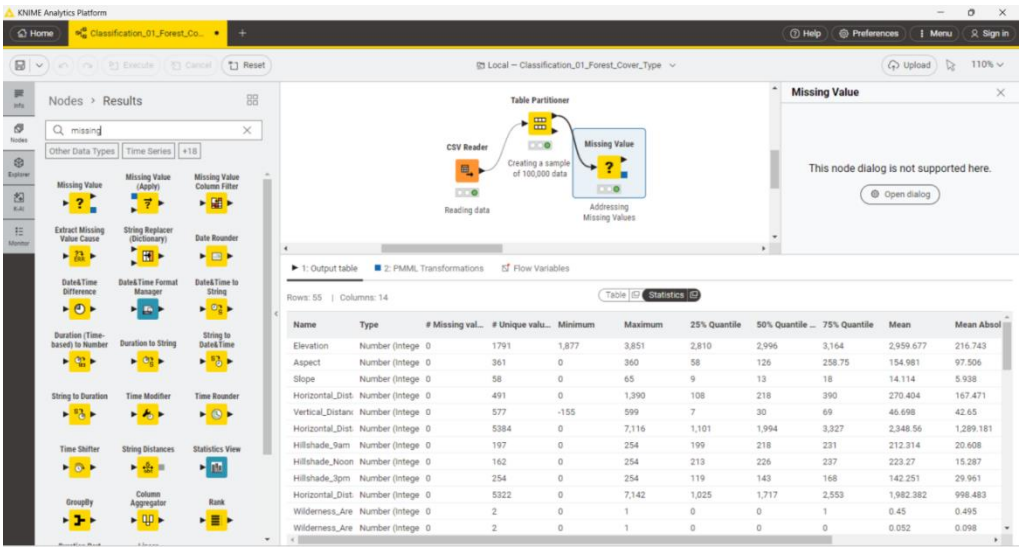


Figure 3: Missing Values Node and Statics Table

No missing values were detected in the sampled dataset, confirmed via the **Missing Value node**.

2.2 Detect and handle outliers

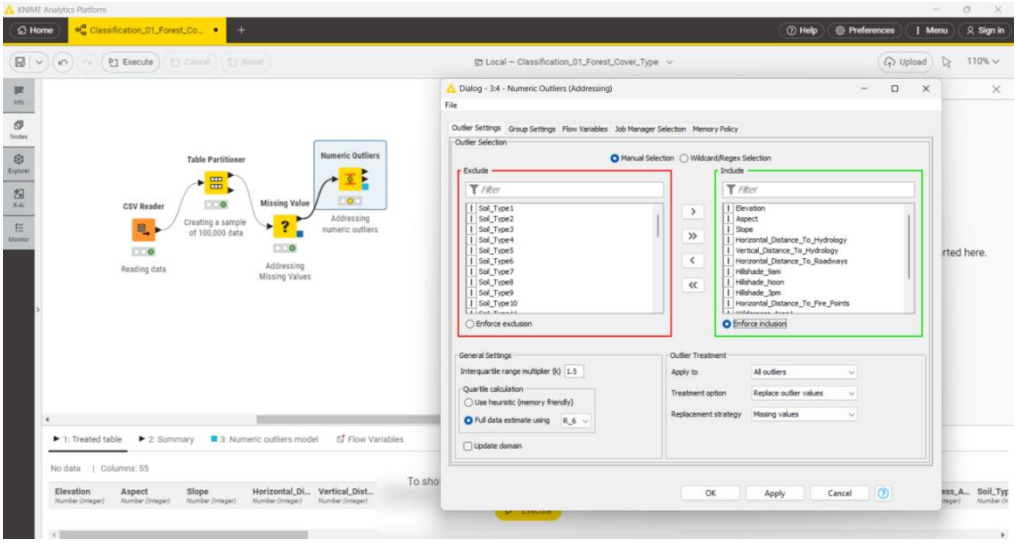


Figure 4: Numeric Outliers Node and Configuration

The **Numeric Outliers node** was applied using the **Inter-Quartile Range (IQR)** method with a multiplier of 1.5.

Detected outlier values in the selected numerical features (Elevation, Aspect, and Slope) were replaced with missing values to minimize the influence of extreme observations.

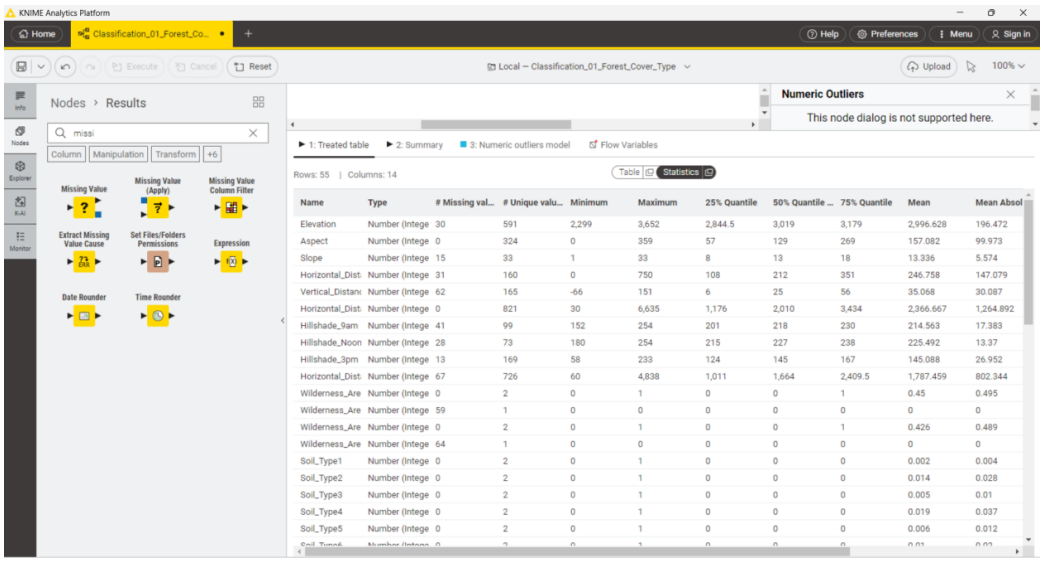


Figure 5: Statistics after Numeric Outliers Node

2.2.1 Why Missing Values Appear After the Numeric Outliers Node

When the Numeric Outliers node is configured to replace outliers with missing values, KNIME flags the detected outliers and substitutes them with “missing” entries instead of deleting the entire row. These missing values represent statistical anomalies that lie far outside the normal range defined by the IQR method. Replacing them with missing entries serves as a placeholder, marking those cells as uncertain or invalid data points.

As a result, the dataset retains the same number of rows, but some numeric cells become missing.

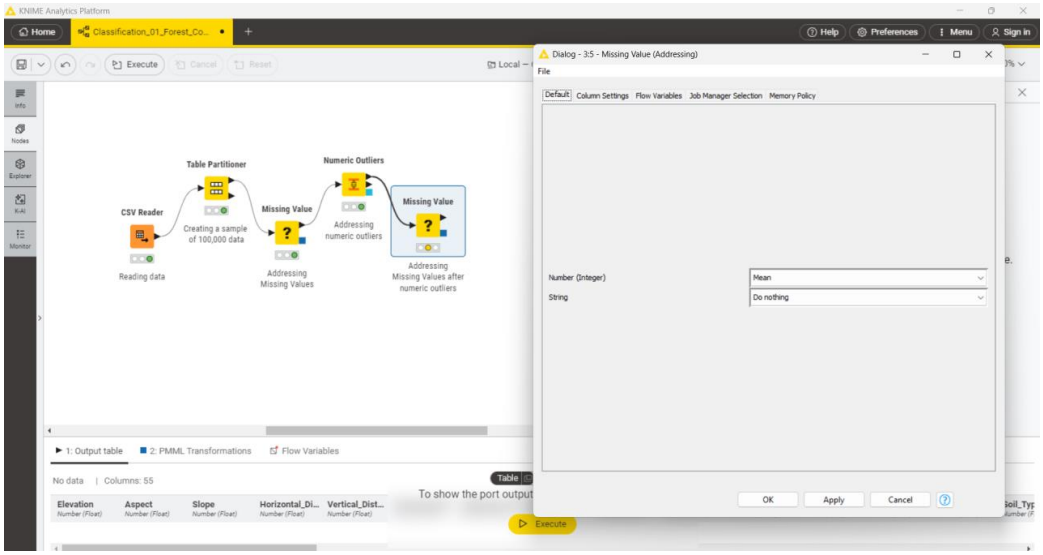


Figure 6: Missing Values Node and Configuration

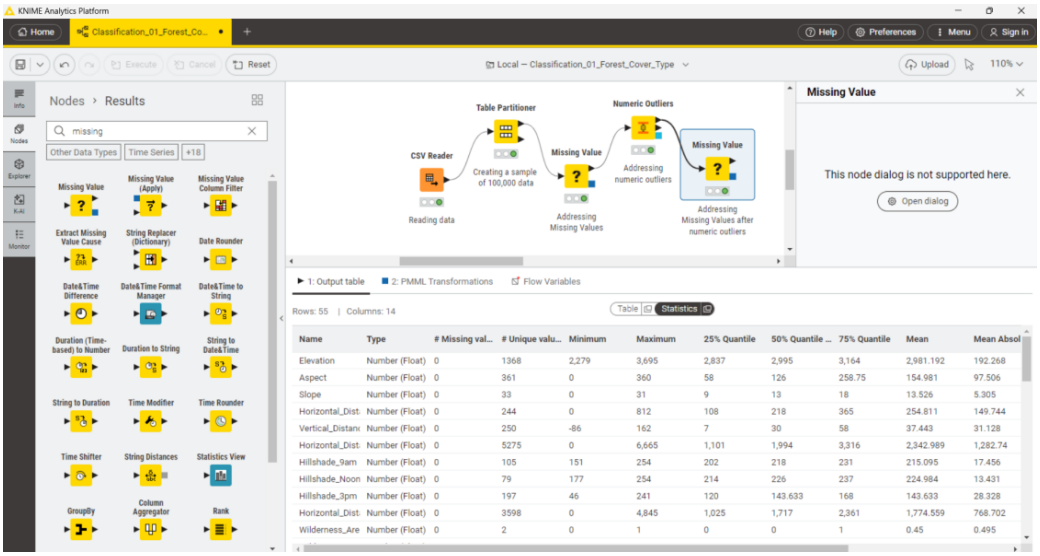


Figure 7: Statistics after Missing Values Node

Adding a **Missing Value node** after the **Numeric Outliers node** imputes these missing entries using methods such as **mean**, median, or most frequent value. This ensures the dataset remains complete for model training, since most machine learning algorithms in KNIME cannot process missing values directly.

2.3 Data transformation: Normalization

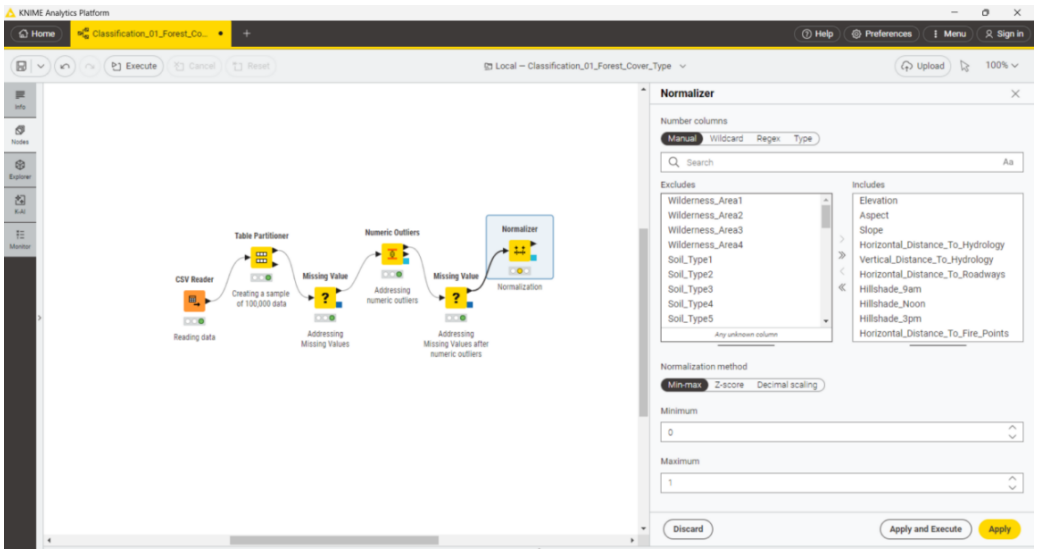


Figure 8: Normalizer Node and Configuration

The ten continuous numeric features (*Elevation, Aspect, Slope, Horizontal and Vertical Distances to Hydrology, Horizontal Distance to Roadways, Hillshade indices at 9 a.m., Noon, and 3 p.m., and Horizontal Distance to Fire Points*) were normalized to the [0, 1] range using the **Min-Max** method in **Normalizer node**. Binary indicator features for Wilderness Areas and Soil Types, as well as the target variable *Cover_Type*, were excluded.

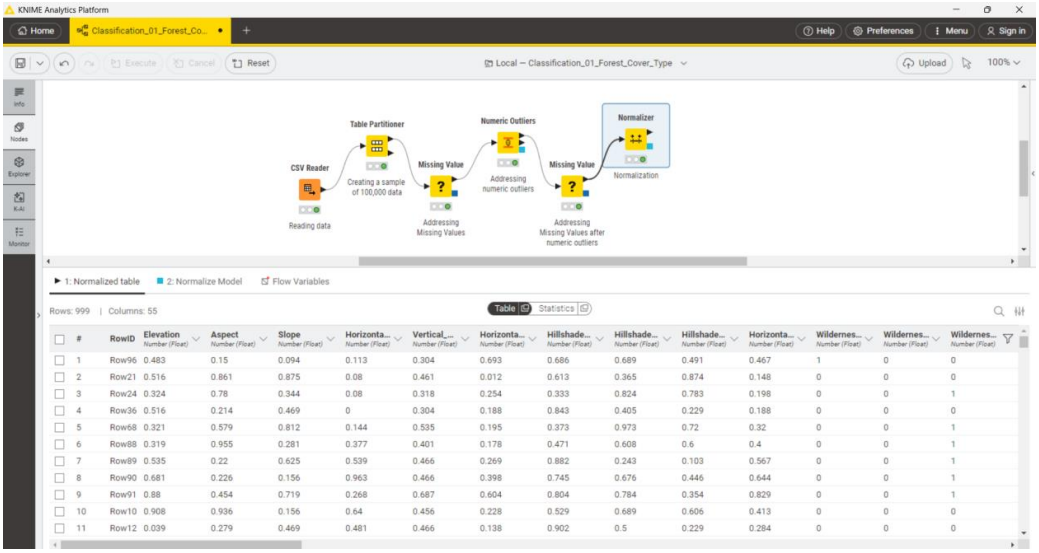


Figure 9: Output Table after Normalizer

Continuous numerical features were scaled to the [0, 1] range using the Normalizer node to ensure equal weight across attributes for distance-based and gradient-based models.

2.4 Feature selection and engineering

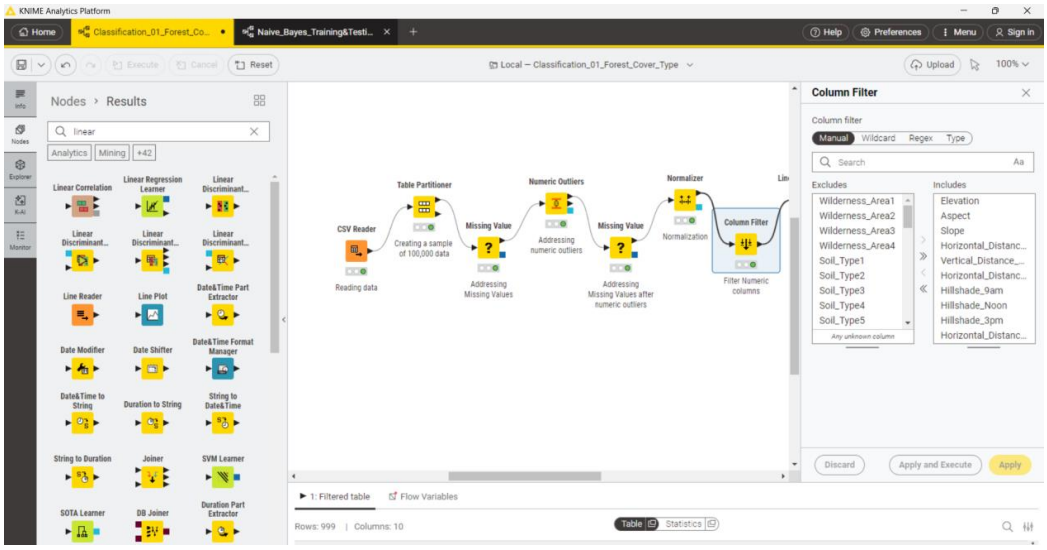


Figure 10: Column Filter Node and Configuration

A Column Filter node was used to restrict the analysis to continuous numerical features only.

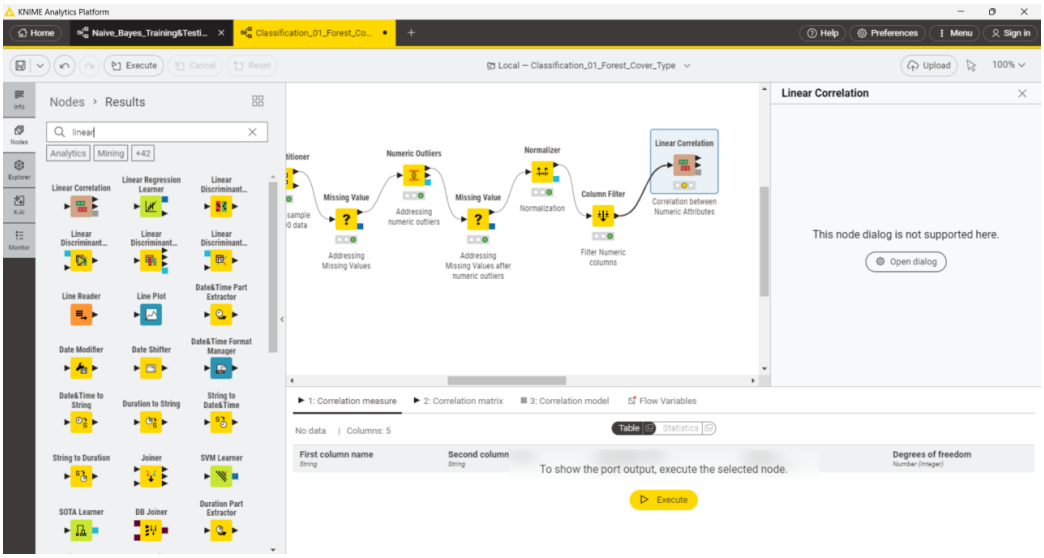


Figure 11: Linear Correlation Node

After normalizing the dataset and selecting only the required features using the **Column Filter node**, the **Linear Correlation node** was used to identify relationships between numeric variables.

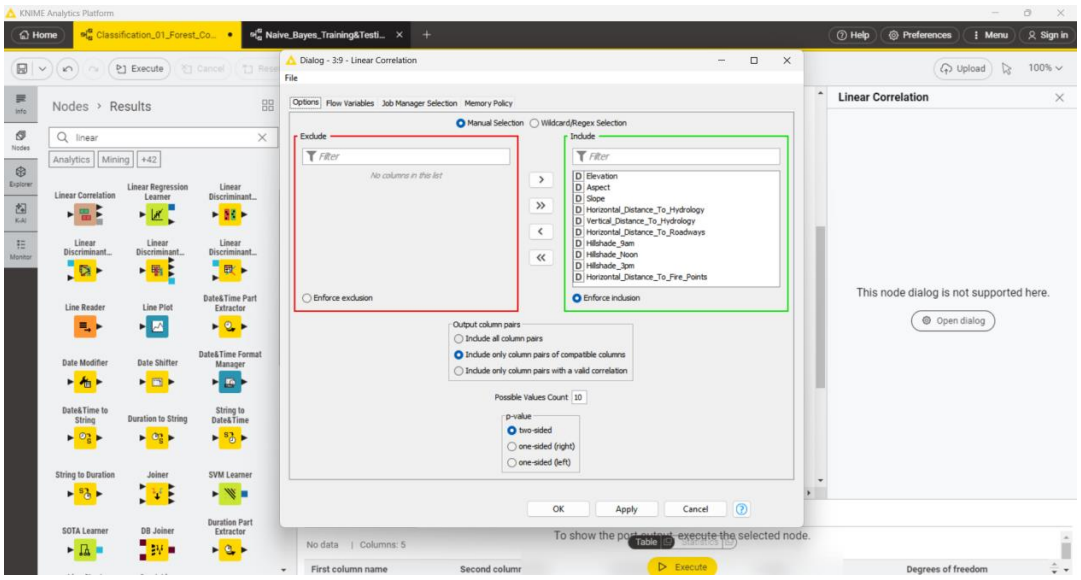


Figure 12: Configuration of Linear Correlation Node

This node measures the linear dependency between pairs of attributes using the Pearson correlation coefficient (r).

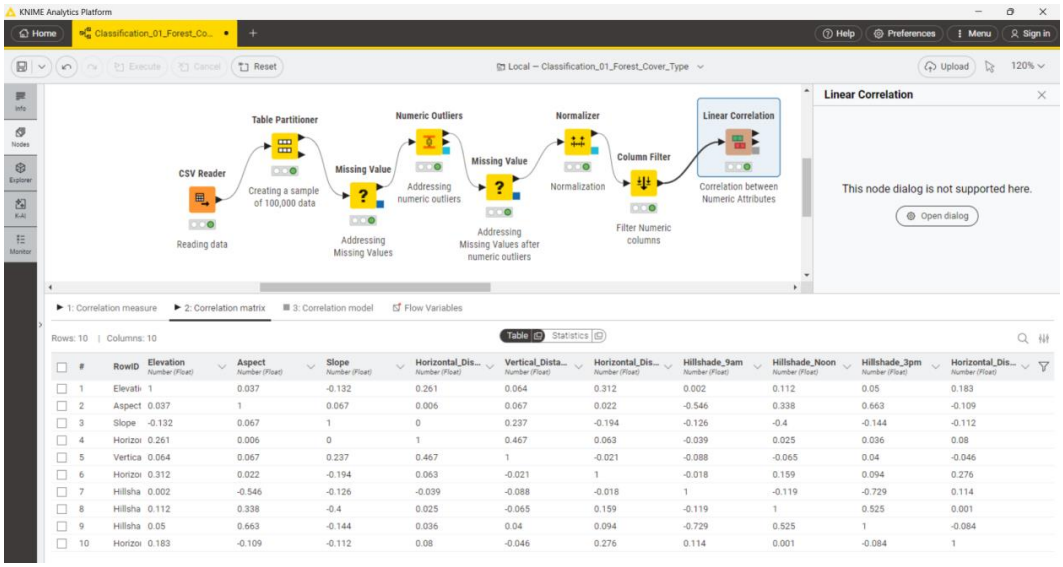


Figure 13: Output Table of Linear Correlation Node - Correlation Metrix

Values of r close to +1 indicate a strong positive relationship, while values near –1 indicate a strong negative relationship.

Features with high correlation provide redundant information, which can negatively affect model performance and increase computational cost.

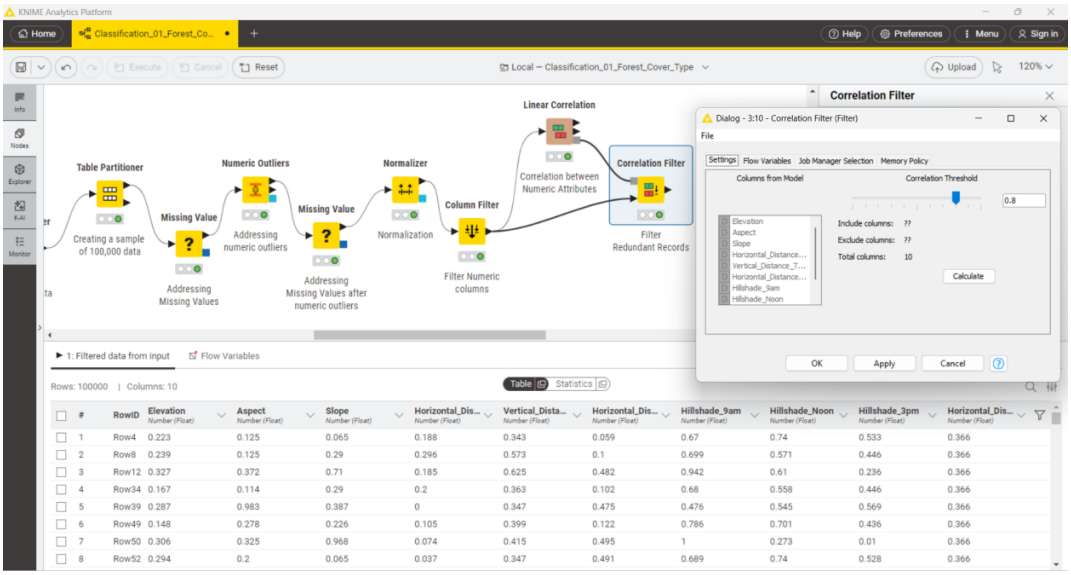


Figure 14: Correlation Filter Node, Configuration and Output Table

The **Correlation Filter** node was configured with a correlation threshold of 0.8 to automatically remove redundant features identified in the **Linear Correlation** model. This ensured that only independent attributes were retained for model training.

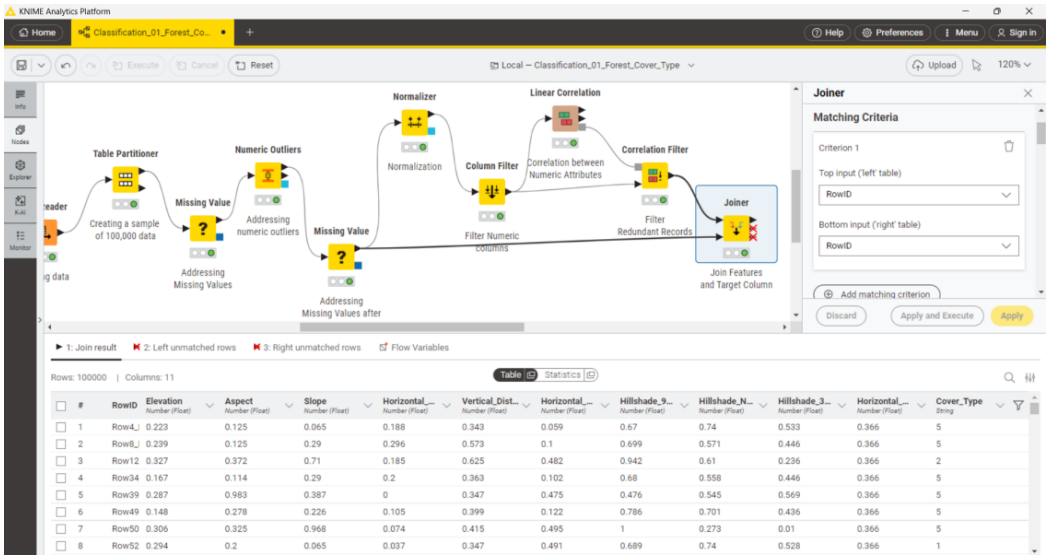


Figure 15: Joiner Node, Configuration, and Output Table

After applying the correlation-based feature selection, a **Joiner node** was used to combine the selected feature set with the target variable, *Cover_Type*.

The join was performed using the *RowID* column as the matching key to ensure that each record was accurately aligned.

In the configuration, the essential features were selected from the top input (feature set), while the *Cover_Type* column was chosen from the bottom input.

Matching rows were included in the output, and merge join columns were enabled to maintain a consistent structure. This step produced the final dataset containing all selected predictors along with the target variable, ready for model training.

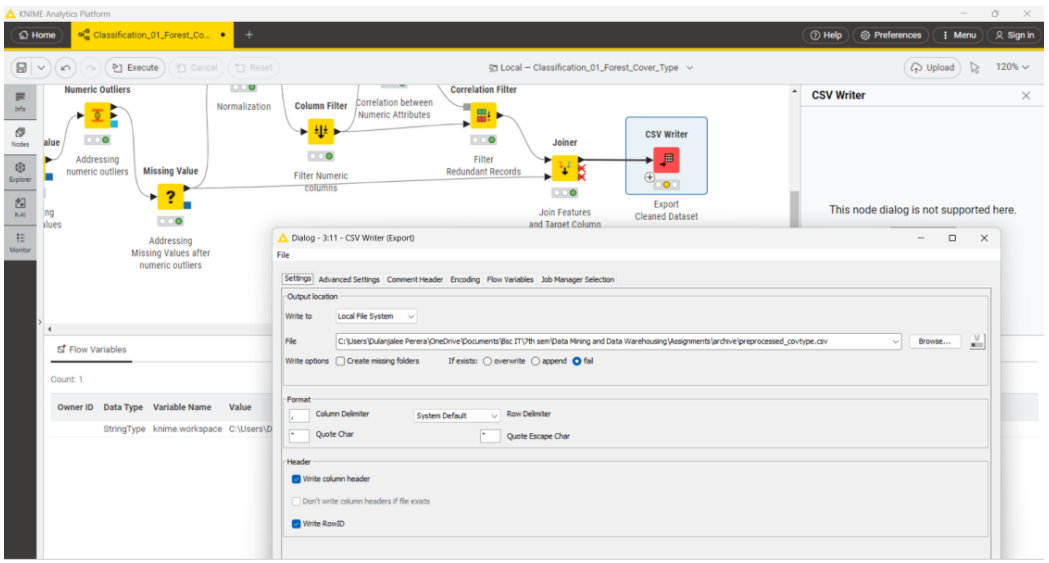


Figure 16: CSV Writer and Configuration

After completing all preprocessing and feature selection steps, the final dataset, containing normalized and non-redundant features, was exported using a **CSV Writer node**.

This exported dataset serves as the standardized input for all model training and testing processes.

Separate KNIME workflows were created for each classification algorithm (*Naïve Bayes, Decision Tree, K-Nearest Neighbors, SVM, and Neural Network*) to maintain workflow clarity, ensure consistent use of the same preprocessed data, and simplify the comparison of model performance.

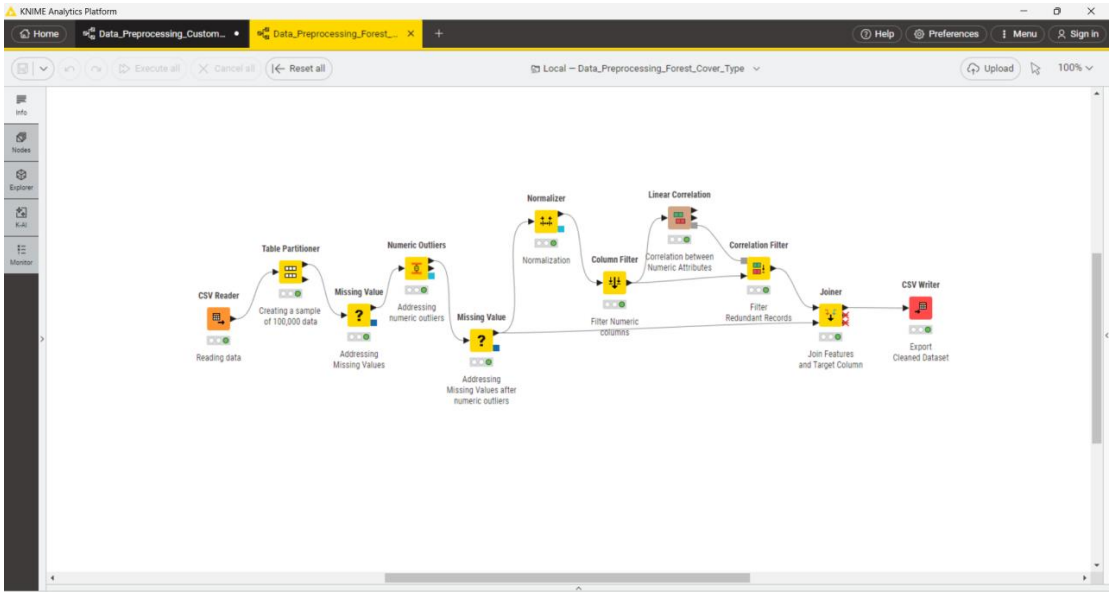


Figure 17: Full Workflow of Data Preprocessing

3. Model Training and Testing

(a) Naive Bayes

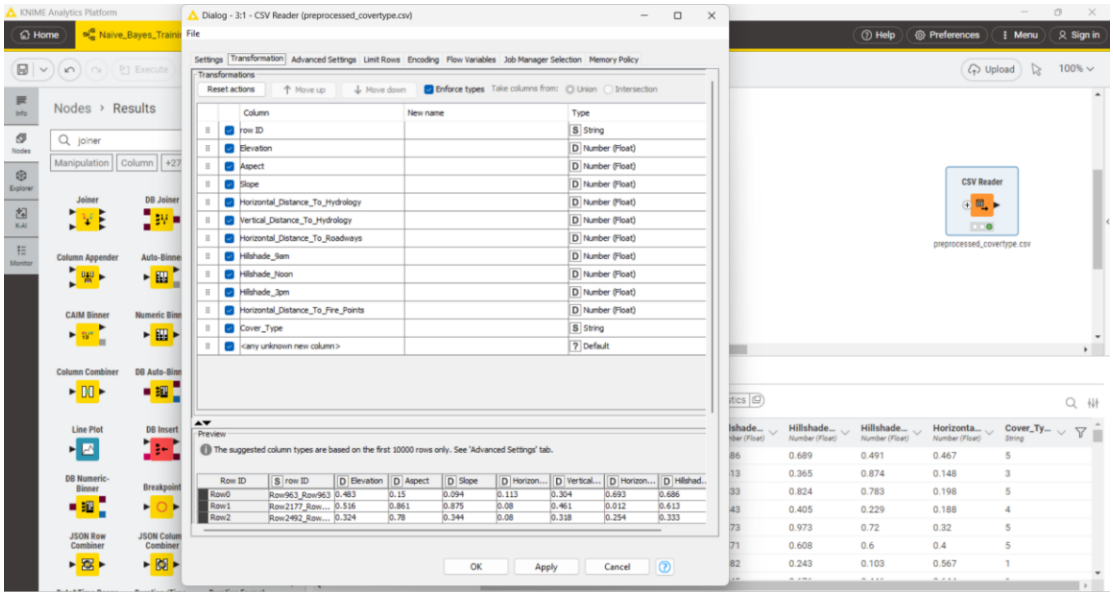


Figure 18: CSV Reader Node, Configuration, and Output Table

During data import, the *Cover_Type* column was configured as a string type within the **CSV Reader** node to ensure it was recognized as a categorical variable suitable for classification.

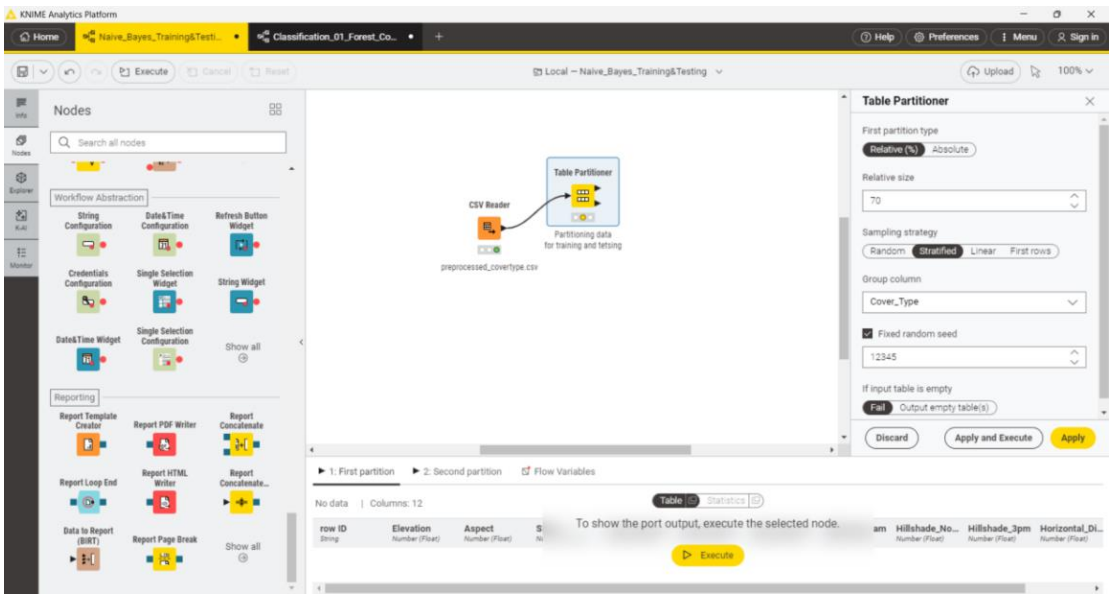


Figure 19: Table Partitioner Node and Configuration

The dataset was divided into training and testing subsets using the **Table Partitioner** node.

A **relative partitioning** method was applied with 70 % of the records allocated for training and 30 % for testing.

Stratified sampling was selected based on the *Cover_Type* column to preserve the proportional representation of each class in both subsets.

This ensured a fair and balanced evaluation of the classification model's performance.

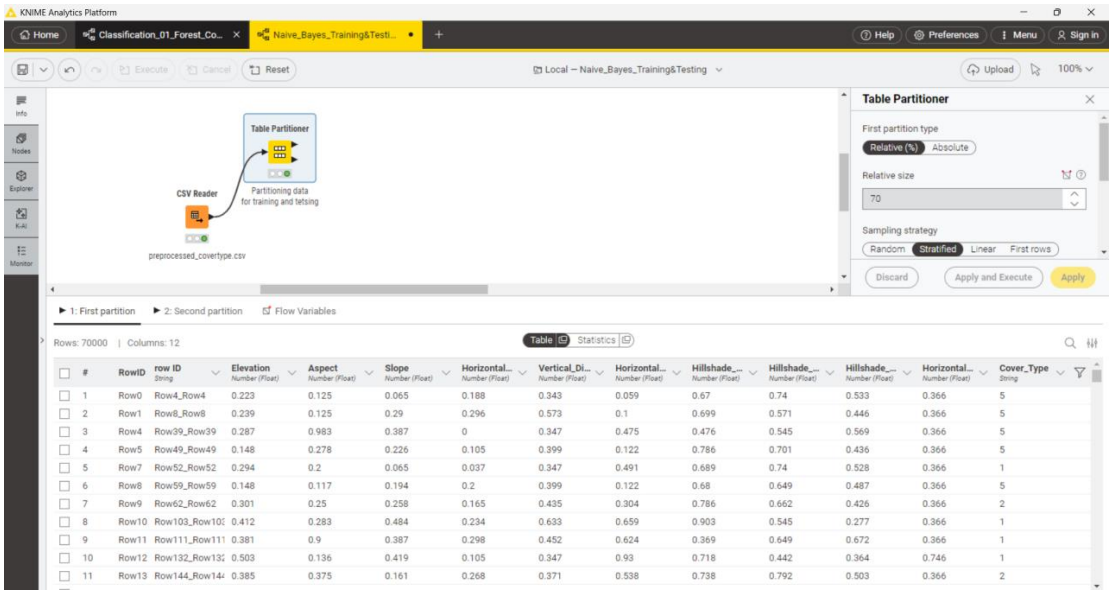


Figure 20: Output Table of Table Partitioner Node - First Partition (70000)

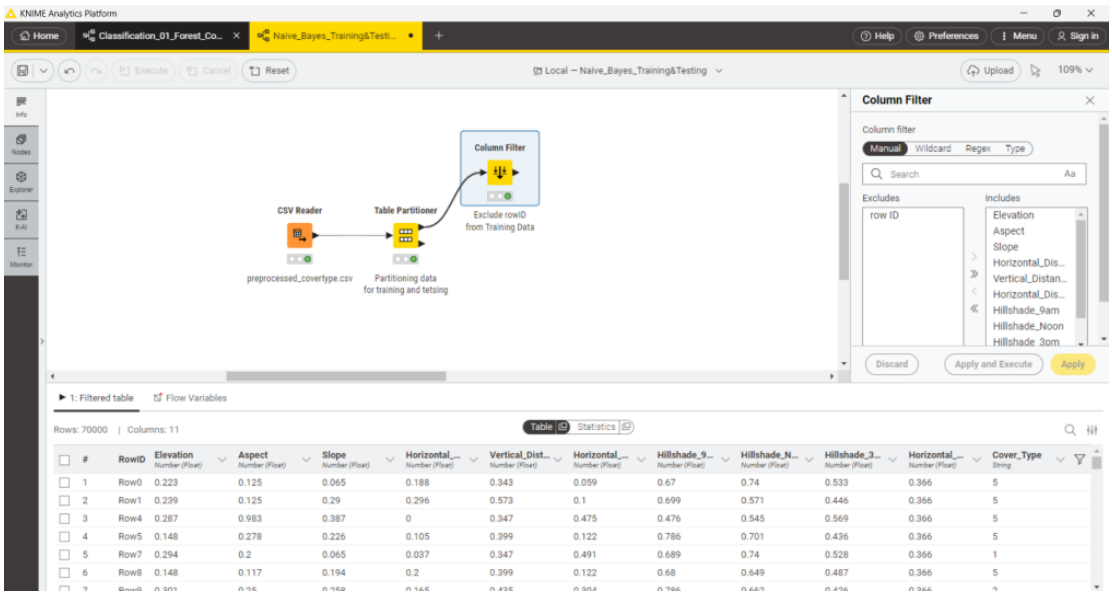


Figure 21: Column Filter Node, Configuration, and Output Table

As Naive Bayes model is a classification algorithm, *row ID* does not participate in classification. To exclude *row ID* from the dataset, a **Column Filter** is used.

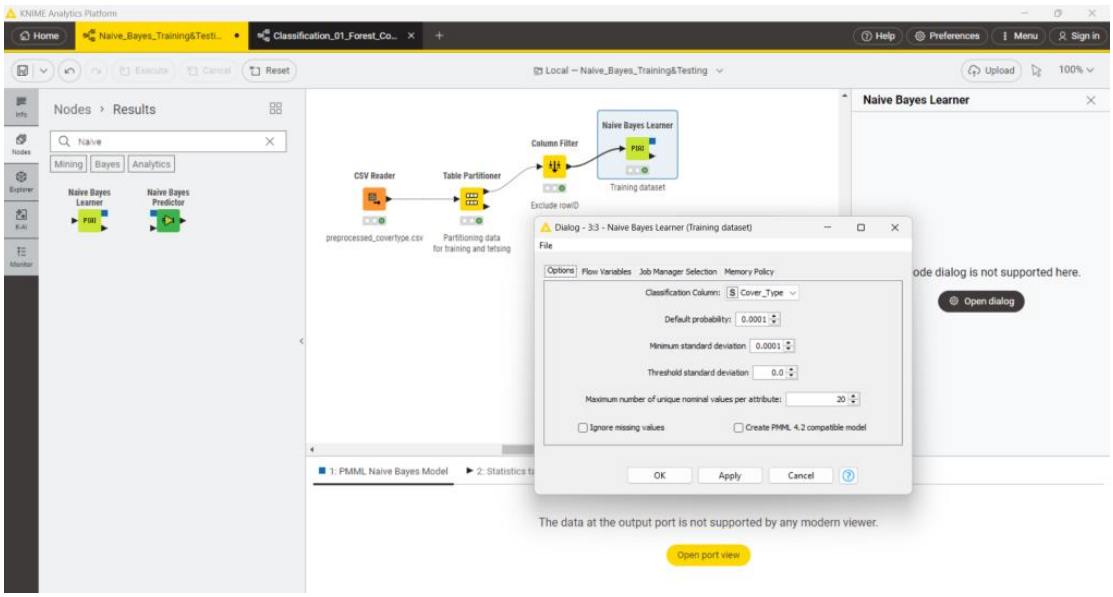


Figure 22: Naive Bayes Learner Node and Configuration

Cover_Type was selected as the classification column, and the rest were set to default. Since this is the training, there is no expected output from the node.

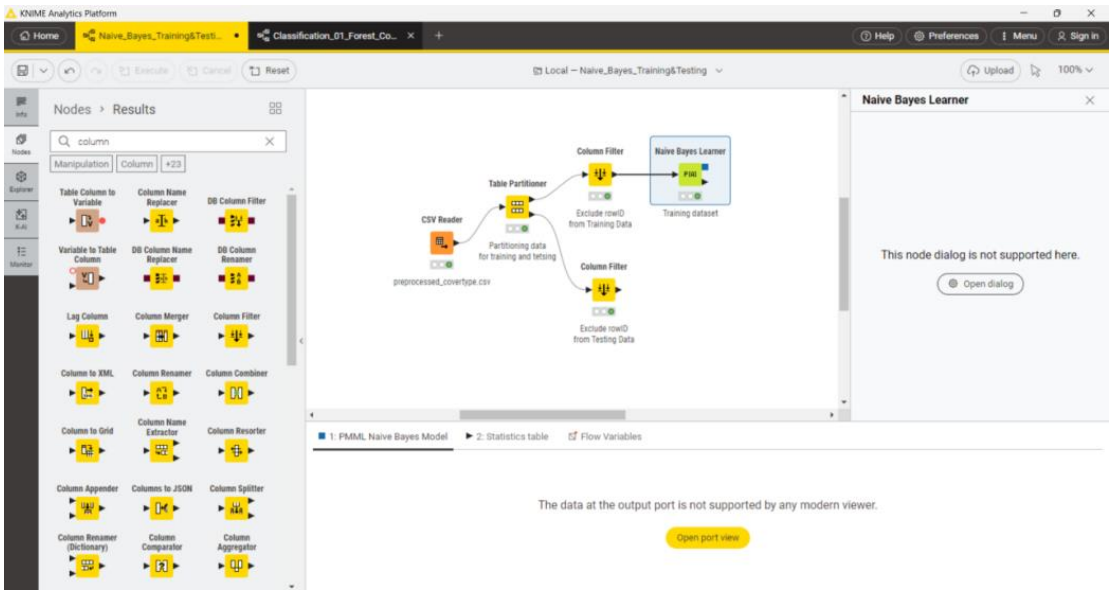


Figure 23: Column Filter Node

Another **Column Filter** was connected to the testing data output of Table Partitioner node in order to exclude *row ID* from the testing dataset sample.

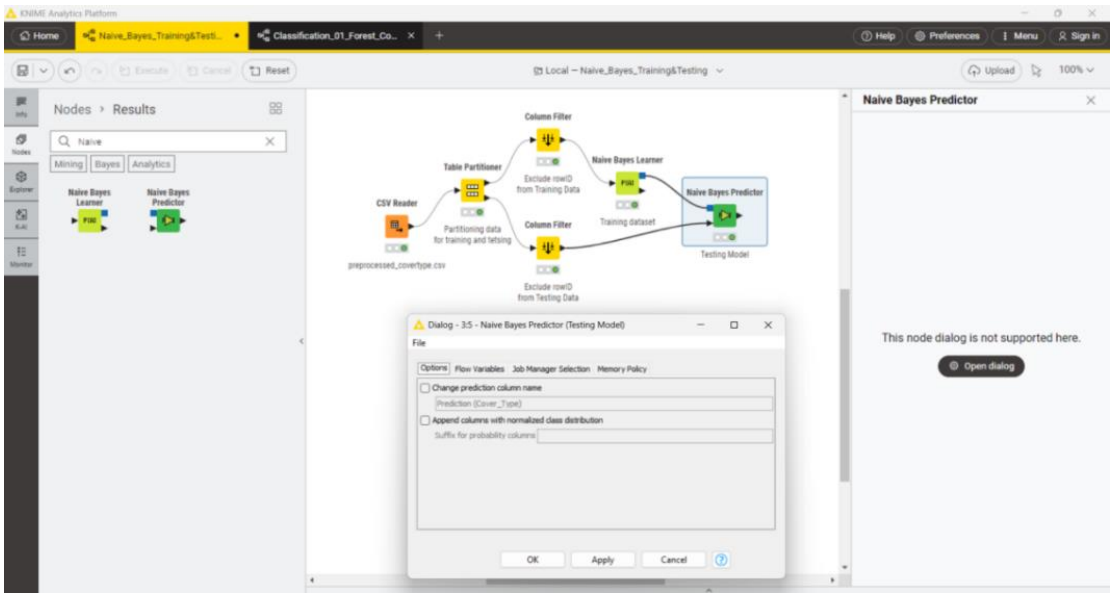


Figure 24: Naive Bayes Predictor Node and Configuration

The configuration was set to default, where *Cover_Type* was already selected as the prediction.

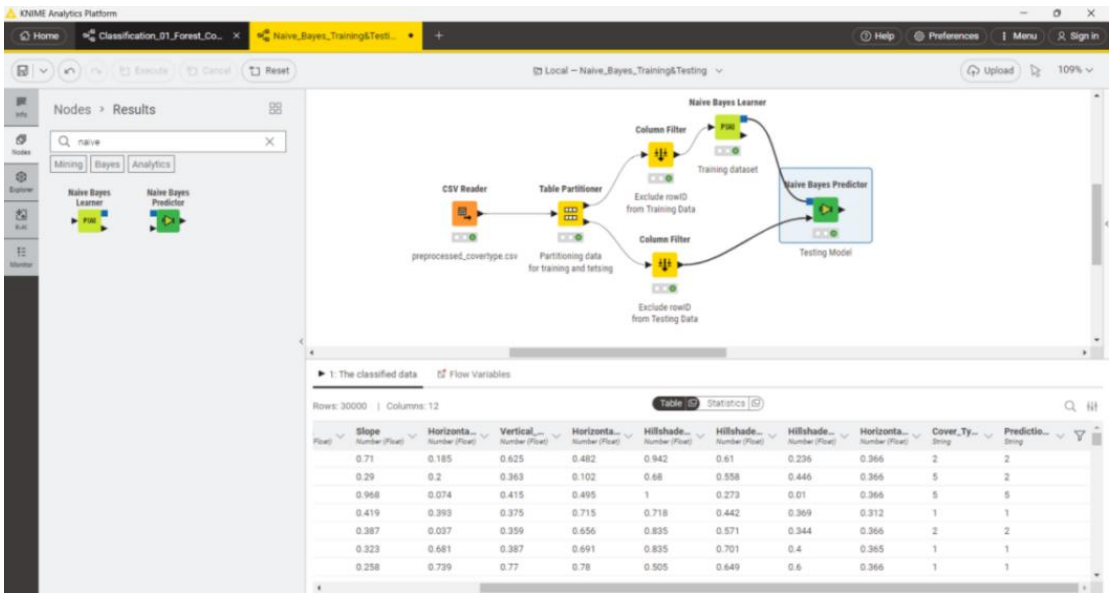


Figure 25: Output Table of Naive Bayes Predictor Node

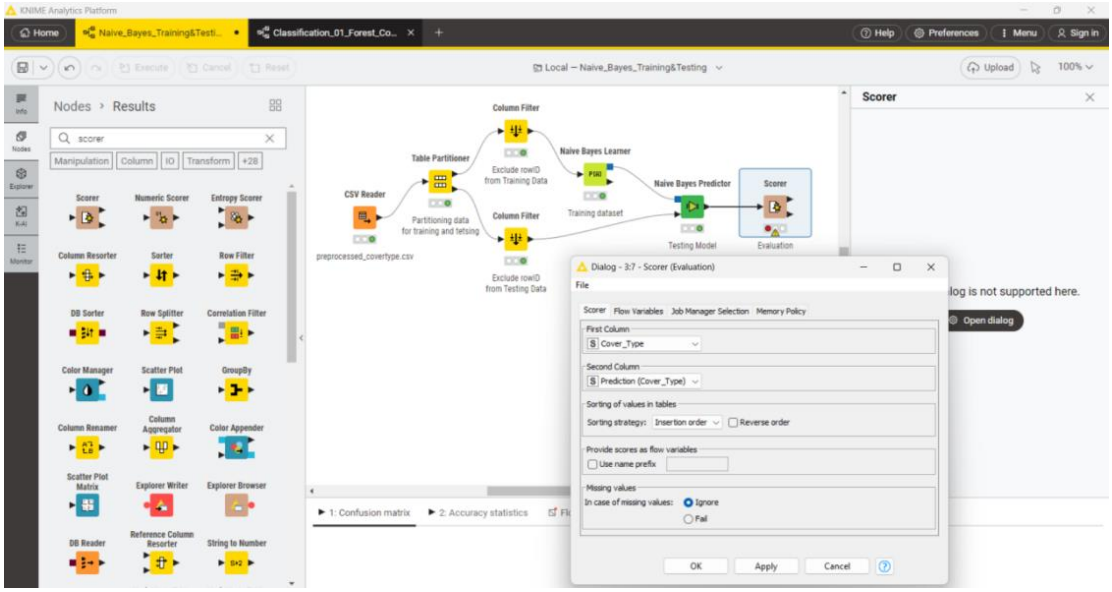


Figure 26: Scorer Node and Configuration

Cover_Type from training was selected as the first column of evaluation, and ***Prediction (Cover_Type)*** from testing was selected as the second column of evaluation.

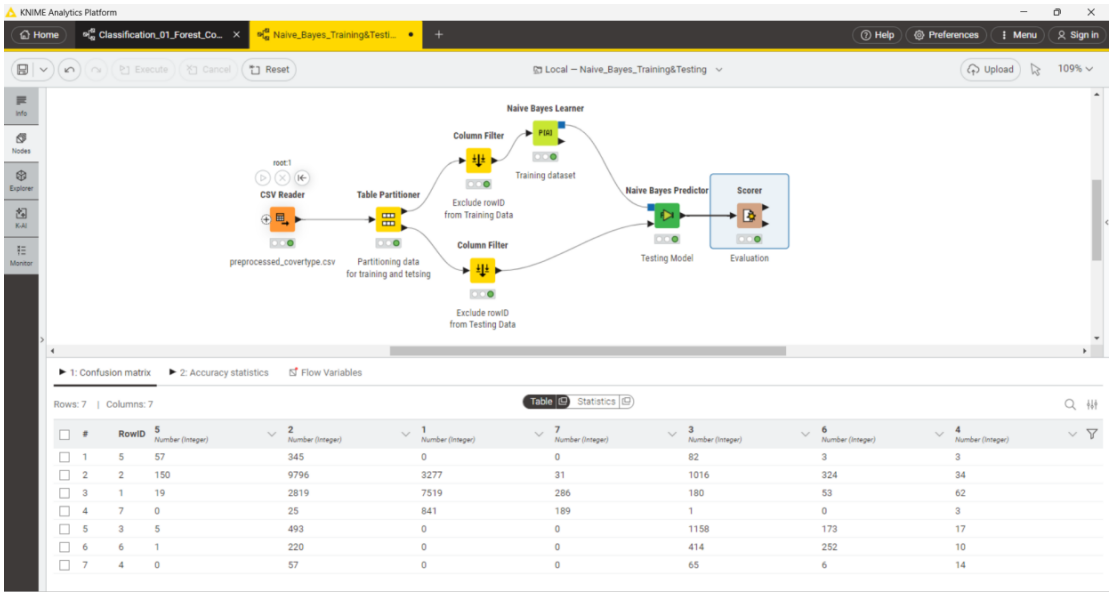
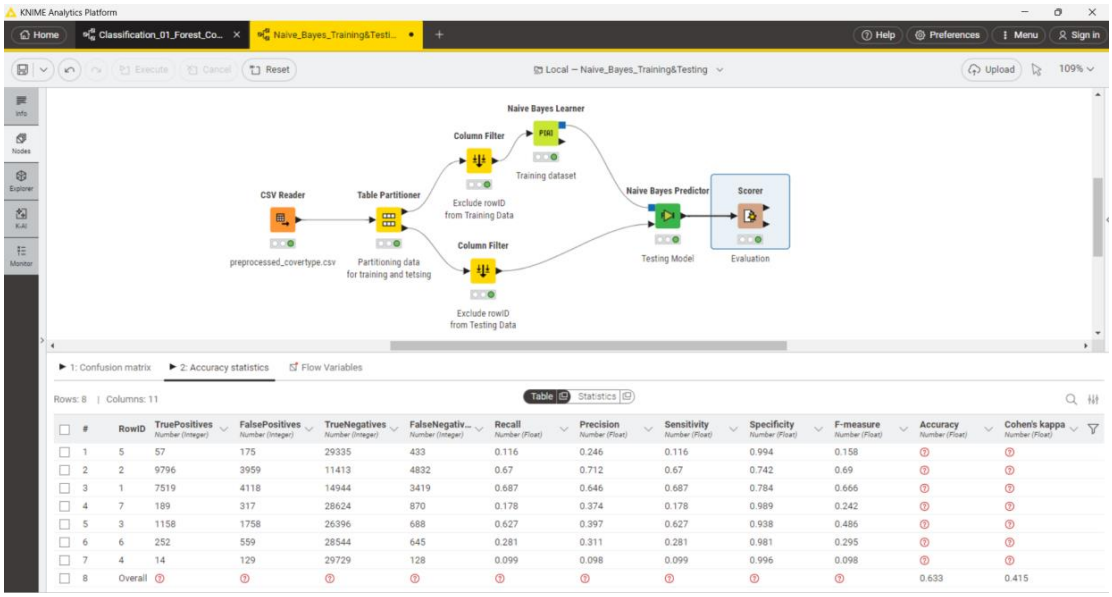


Figure 27: Output of Scorer Node - Confusion Matrix



#	RowID	TruePositives	FalsePositives	TrueNegatives	FalseNegativ...	Recall	Precision	Sensitivity	Specificity	F-measure	Accuracy	Cohen's kappa
1	5	57	175	29335	433	0.116	0.246	0.116	0.994	0.158		
2	2	9796	3959	11413	4832	0.67	0.712	0.67	0.742	0.69		
3	1	7519	4118	14944	3419	0.687	0.646	0.687	0.784	0.666		
4	7	189	317	28624	870	0.178	0.374	0.178	0.989	0.242		
5	3	1158	1758	26396	688	0.627	0.397	0.627	0.938	0.486		
6	6	252	559	28544	645	0.281	0.311	0.281	0.981	0.295		
7	4	14	129	29729	128	0.099	0.098	0.099	0.996	0.098		
8	Overall										0.633	0.415

Figure 28: Output of Scorer Node - Accuracy Statistics

A Naive Bayes classifier was implemented using a dedicated KNIME workflow.

The preprocessed dataset was first read from the exported CSV file and split into 70% training and 30% testing sets using **stratified sampling** on the *Cover_Type* class label.

A **Naive Bayes Learner node** was configured with *Cover_Type* as the target attribute, and predictions were generated using the **Naive Bayes Predictor node**.

The **Scorer node** was then applied to evaluate model performance in terms of accuracy, precision, recall, and F-measure, along with the confusion matrix.

(b) Decision Tree

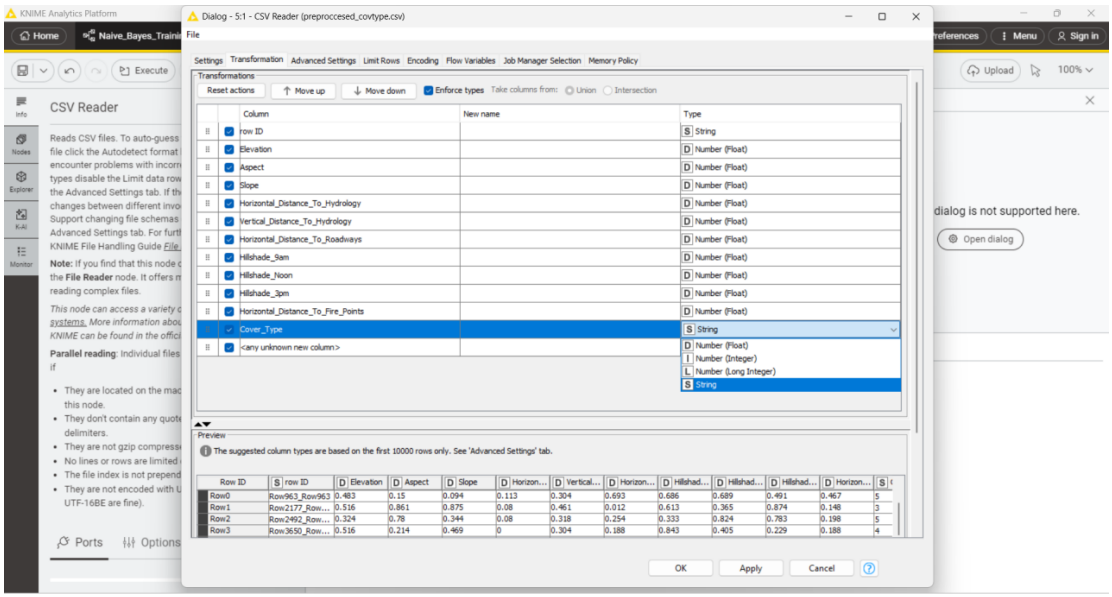


Figure 29: CSV Reader Node Configuration

During data import, the *Cover_Type* column was configured as a string type within the **CSV Reader node** to ensure it was recognized as a categorical variable suitable for classification.

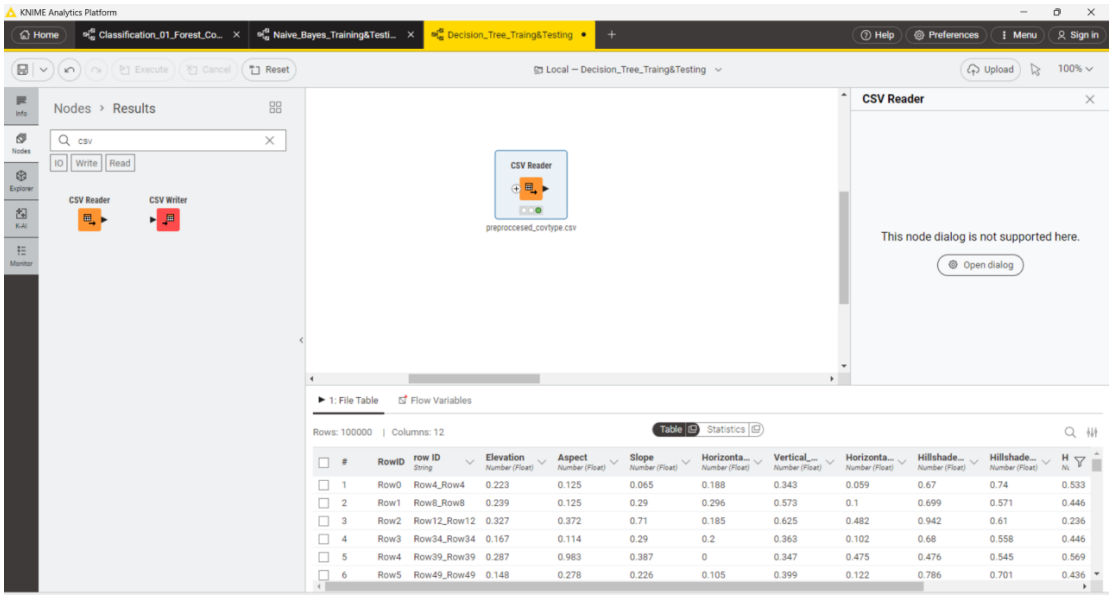


Figure 30: Output of CSV Reader Node

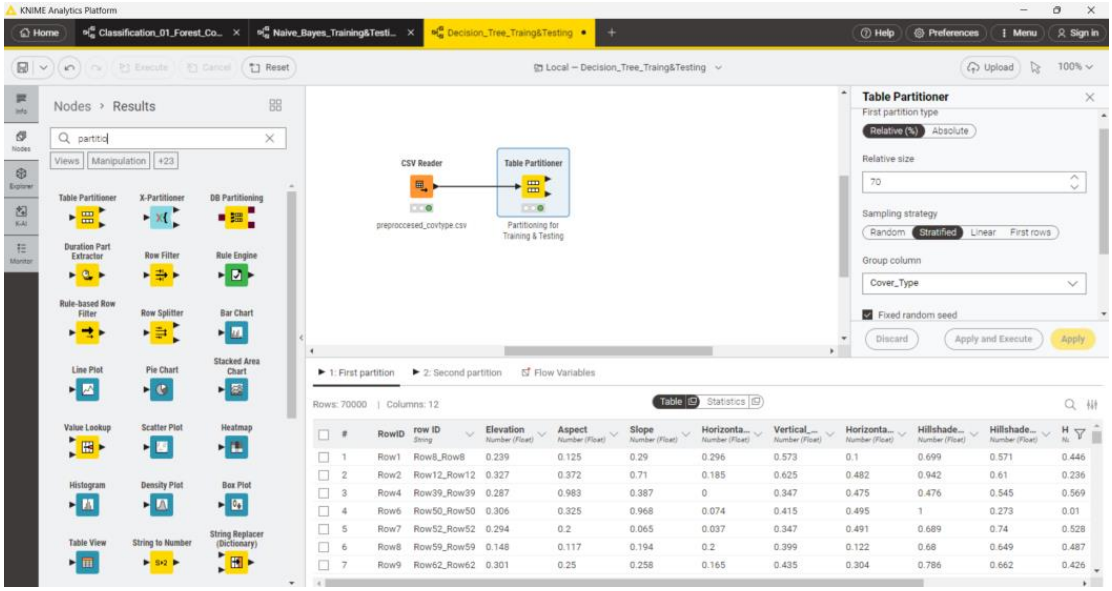


Figure 31: Table Partitioner Node, Configuration and Output - First Partition (70000)

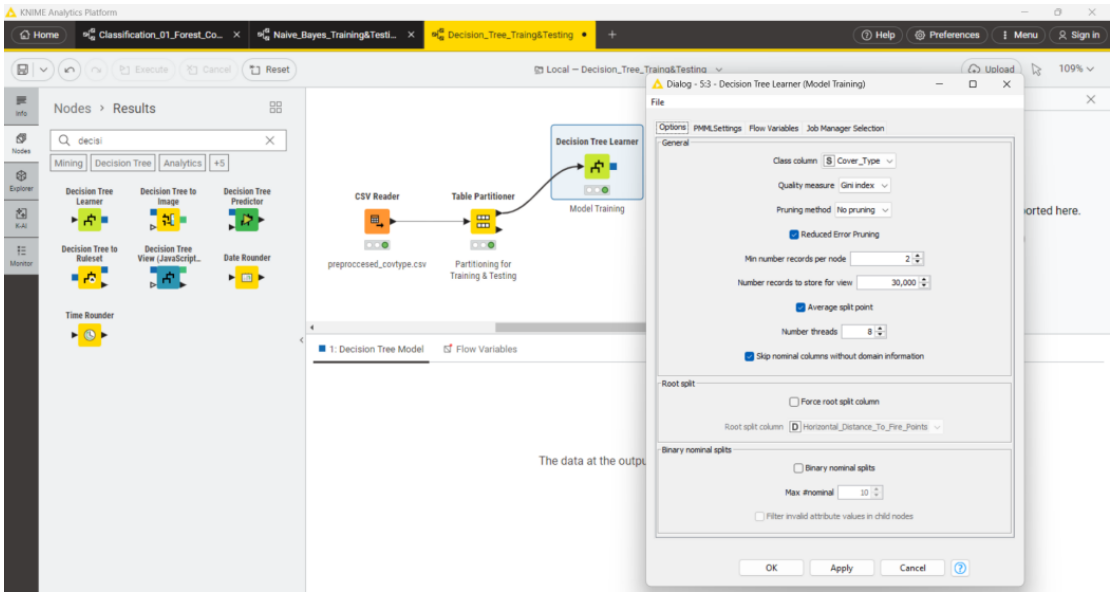


Figure 32: Decision Tree Learner Node, Configuration

Since Decision Tree Learner Node only does the training, there is no expected output from the node.

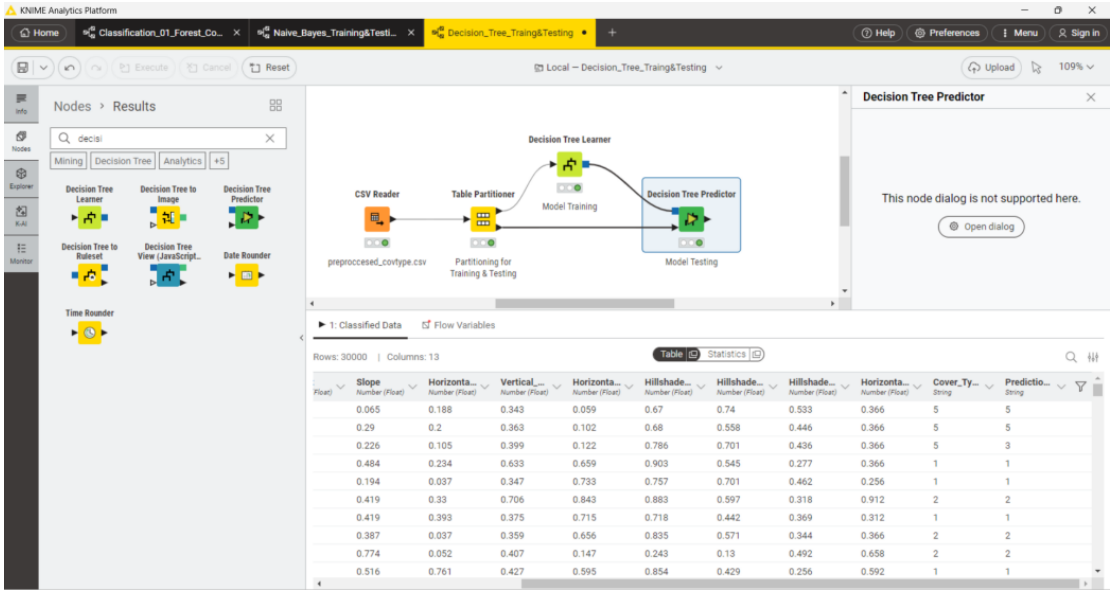


Figure 33: Decision Tree Predictor Node and Output Table

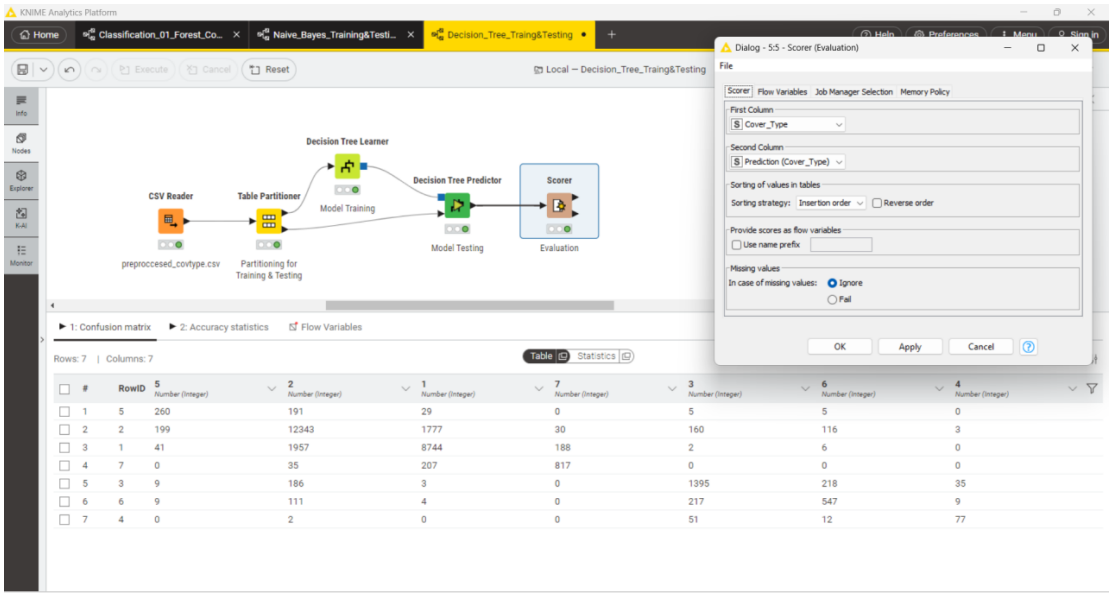


Figure 34: Scorer Node, Configuration, and Output Table - Confusion Matrix

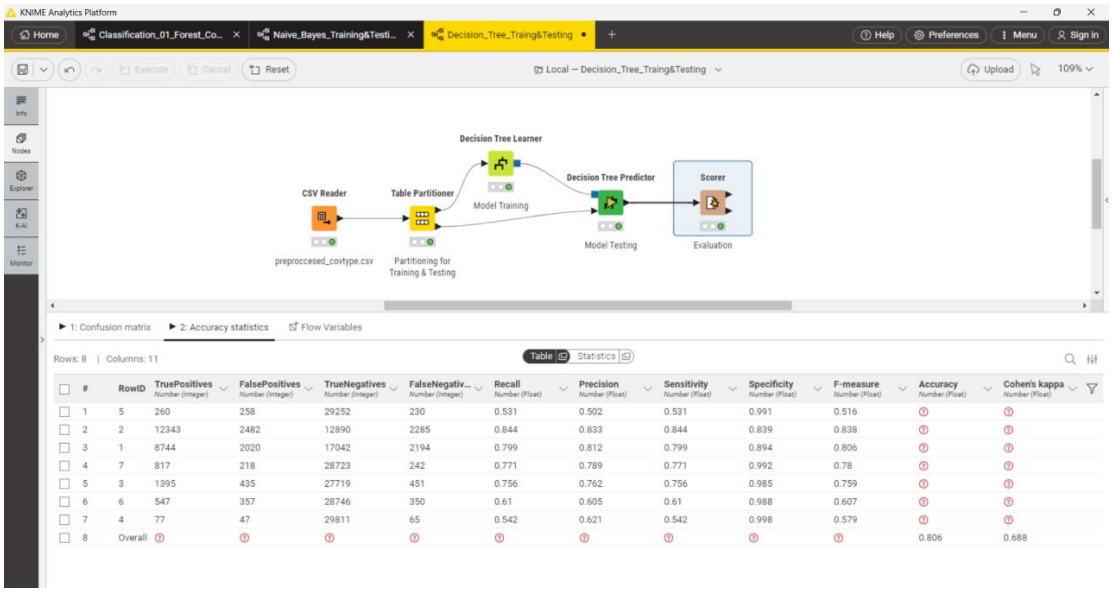


Figure 35: Scorer Node and Output Table - Accuracy Statistics

A **Decision Tree classifier** was developed using the preprocessed dataset.

The data was split into 70 % training and 30 % testing sets using **stratified sampling** based on the *Cover_Type* attribute.

A **Decision Tree Learner node** was configured with *Cover_Type* as the target variable and the **Gini Index** as the splitting criterion.

The trained model was applied to the test set using the **Decision Tree Predictor node**, and performance metrics including accuracy, precision, recall, F-measure, and a confusion matrix were obtained using the **Scorer node**.

(c) K-Nearest Neighbors (KNN)

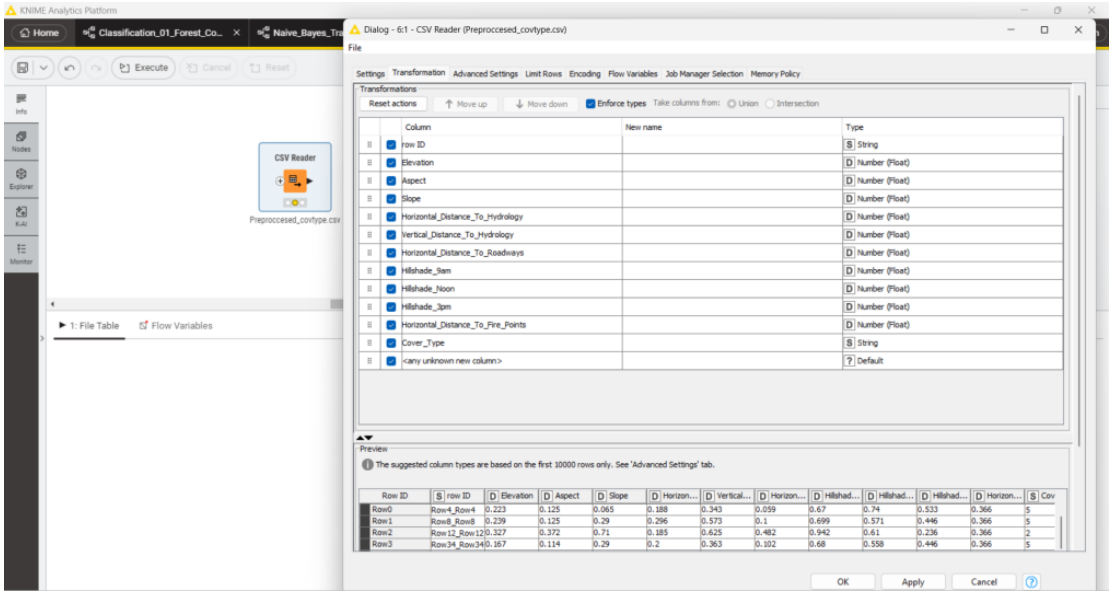


Figure 36: CSV Reader Node and Configuration

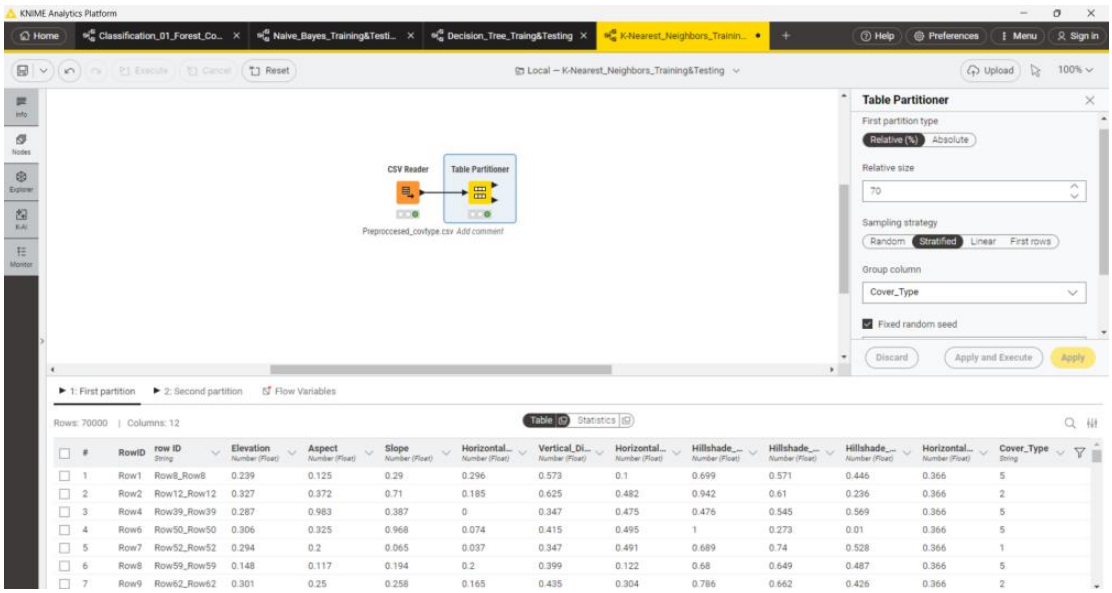


Figure 37: Table Partitioner Node, Configuration, and First Partition (70000)

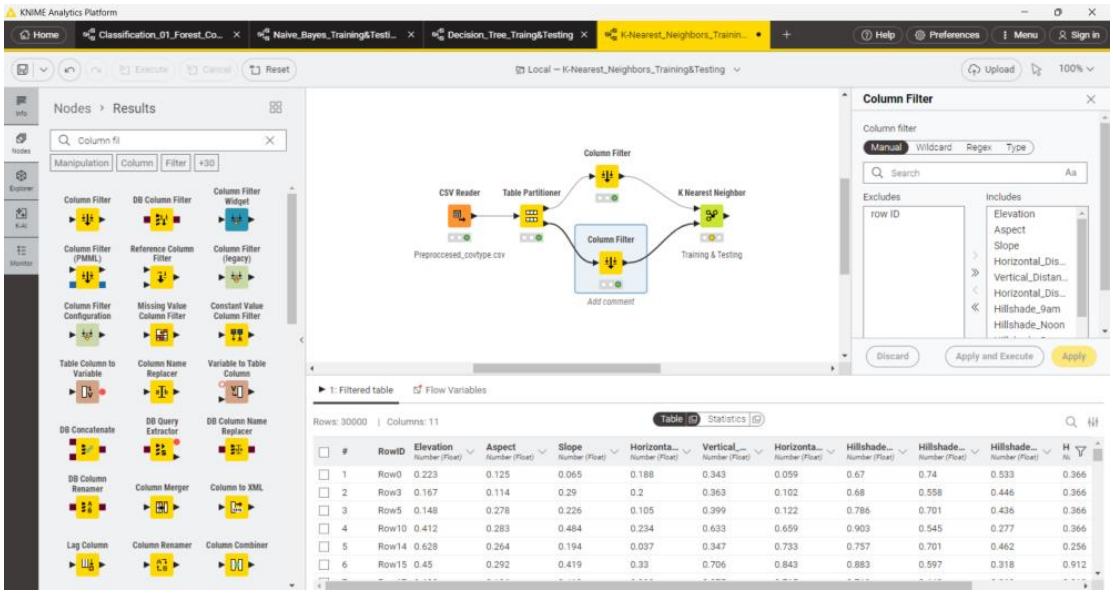


Figure 38: Column Filter Nodes, Configuration, and Output

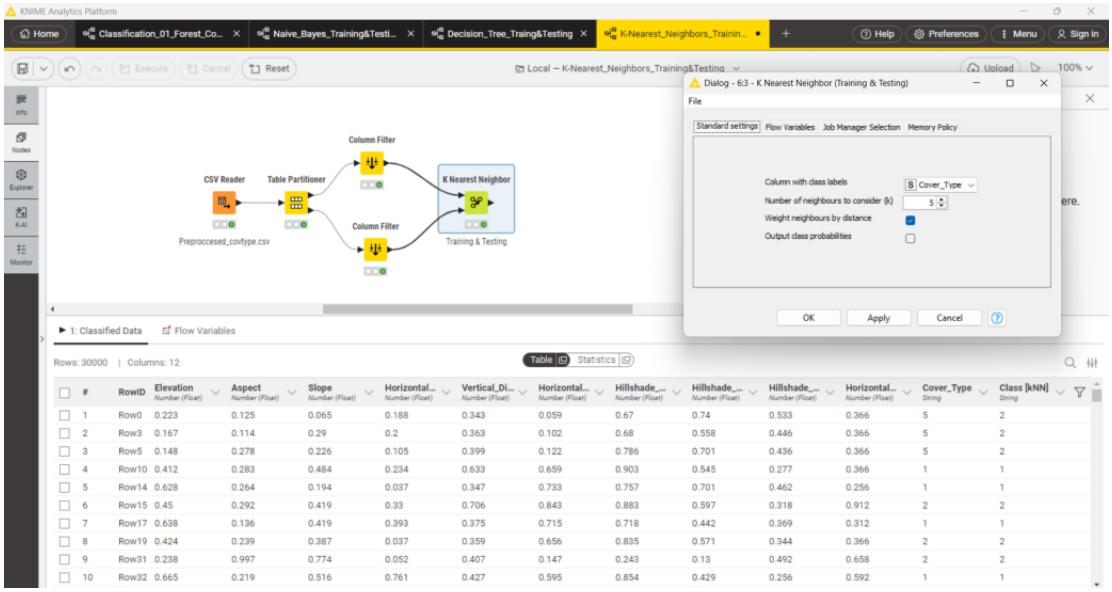


Figure 39: K Nearest Neighbor Node, Configuration, and Output Table

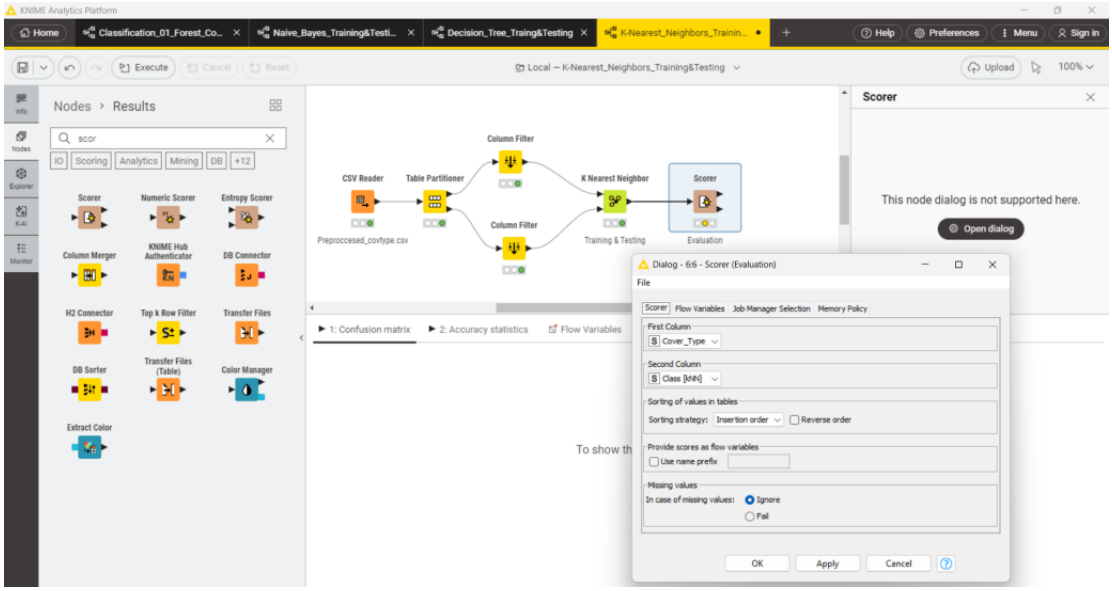


Figure 40: Scorer Node and Configuration

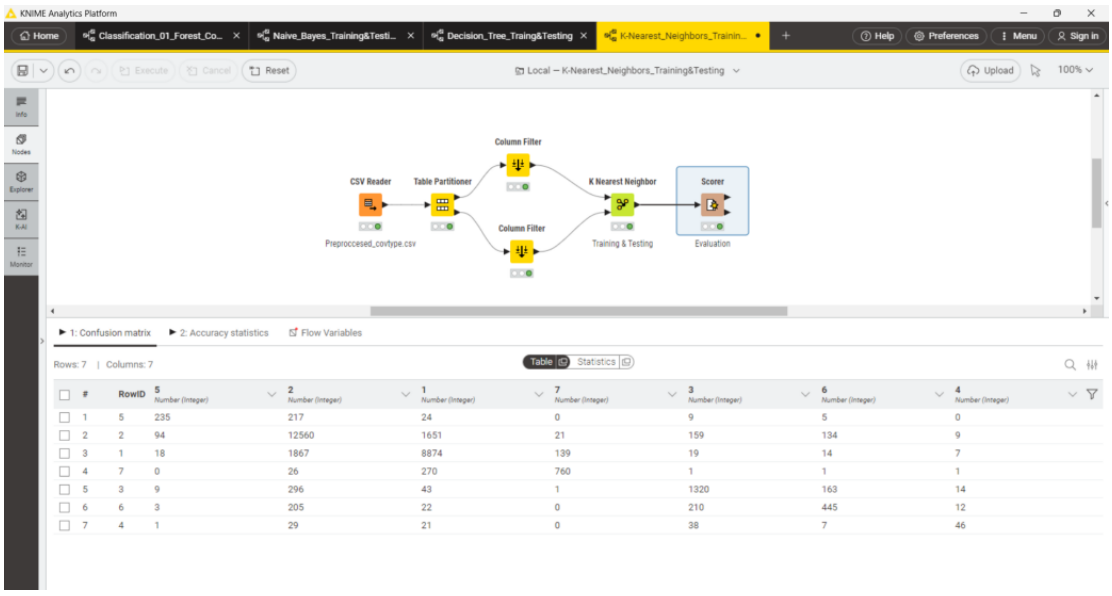


Figure 41: Scorer Node and Output Table - Confusion Matrix

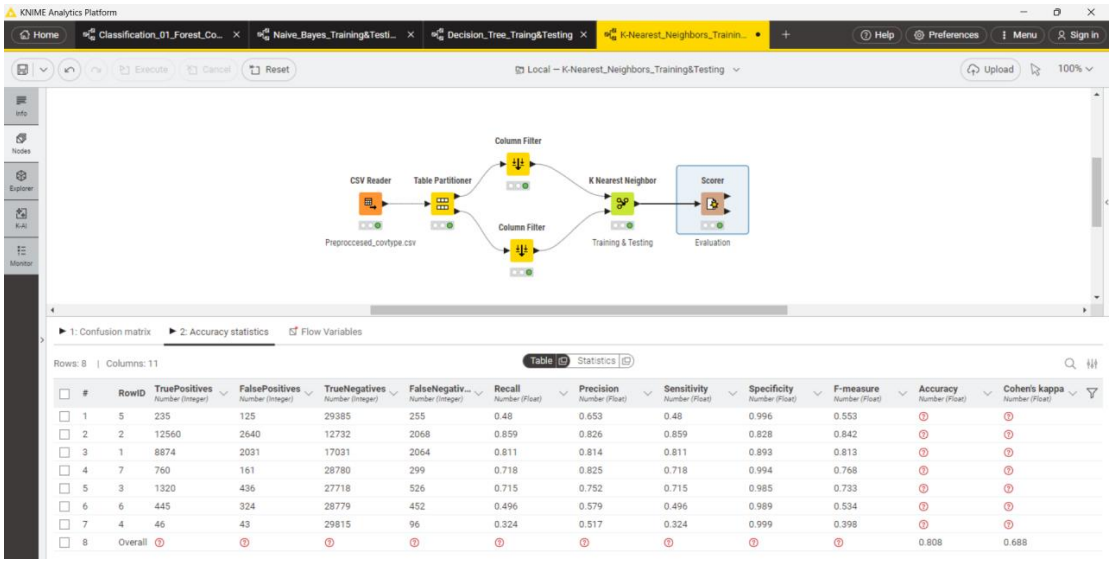


Figure 42: Scorer Node and Output Table - Accuracy Statistics

A **K-Nearest Neighbors (KNN)** classifier was developed using the preprocessed dataset.

The data was divided into 70% training and 30% testing using **stratified sampling** to maintain balanced representation across the *Cover_Type* classes.

The **KNN node** was configured with *Cover_Type* as the target variable, using **Euclidean distance** as the similarity measure, which is appropriate for the normalized numerical attributes in the dataset.

A value of **k = 5** was selected to provide a balance between noise sensitivity and overly smooth decision boundaries.

The combined **K Nearest Neighbor node** was used to perform both training and prediction within a single step, and evaluation metrics such as accuracy, precision, recall, F-measure, and the confusion matrix were generated using the **Scorer node**.

(d) Support Vector Machine (SVM)

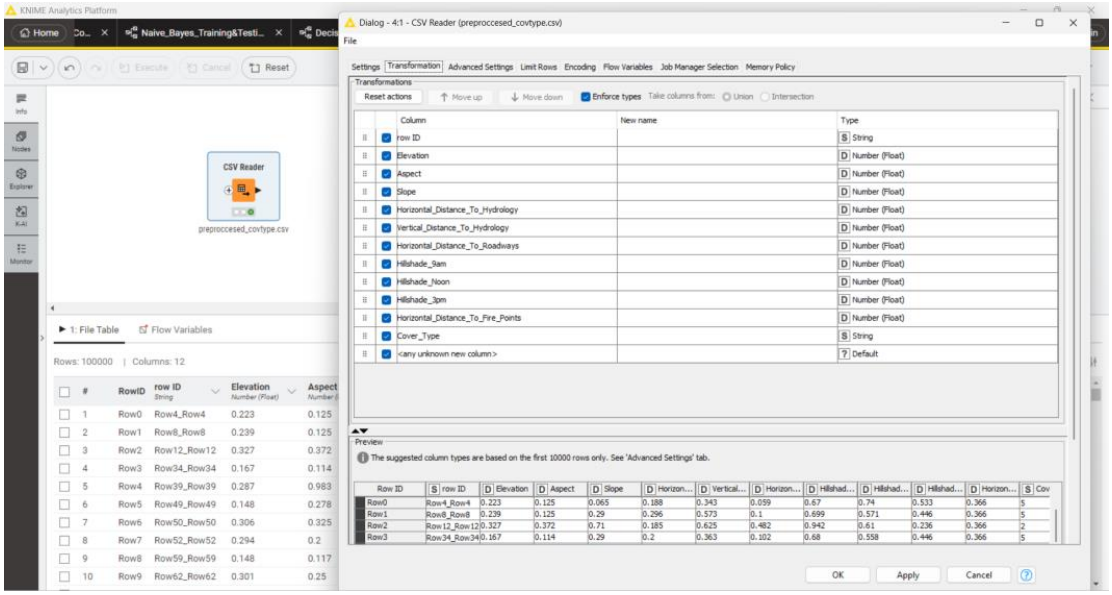


Figure 43: CSV Reader Node, Configuration, and Output Table

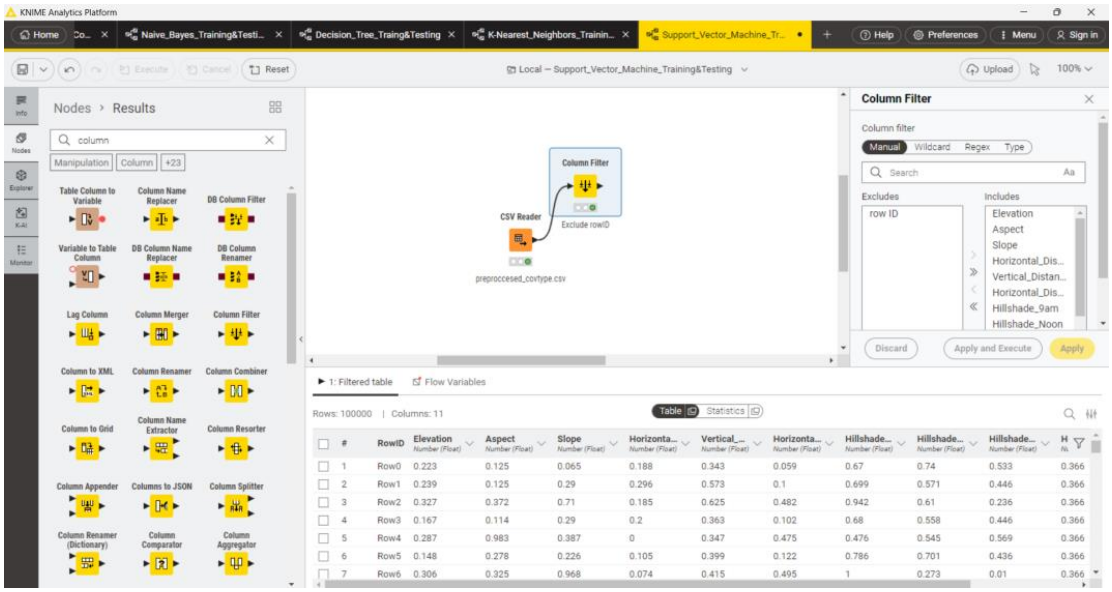


Figure 44: Column Filter Node, Configuration, and Output Table

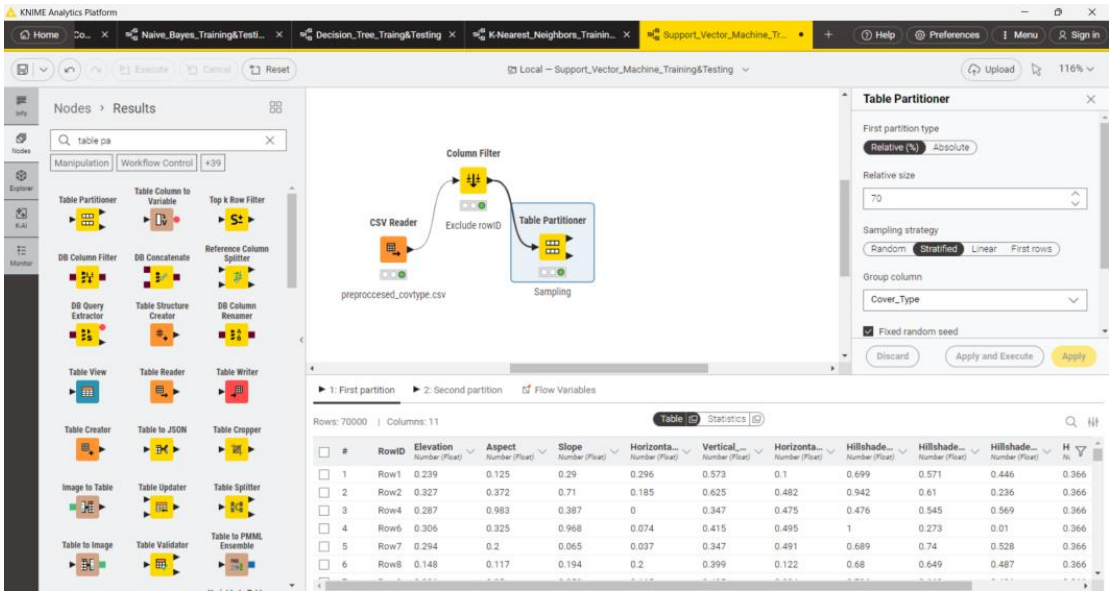


Figure 45: Table Partitioner Node, Configuration, and First Partition (70000)

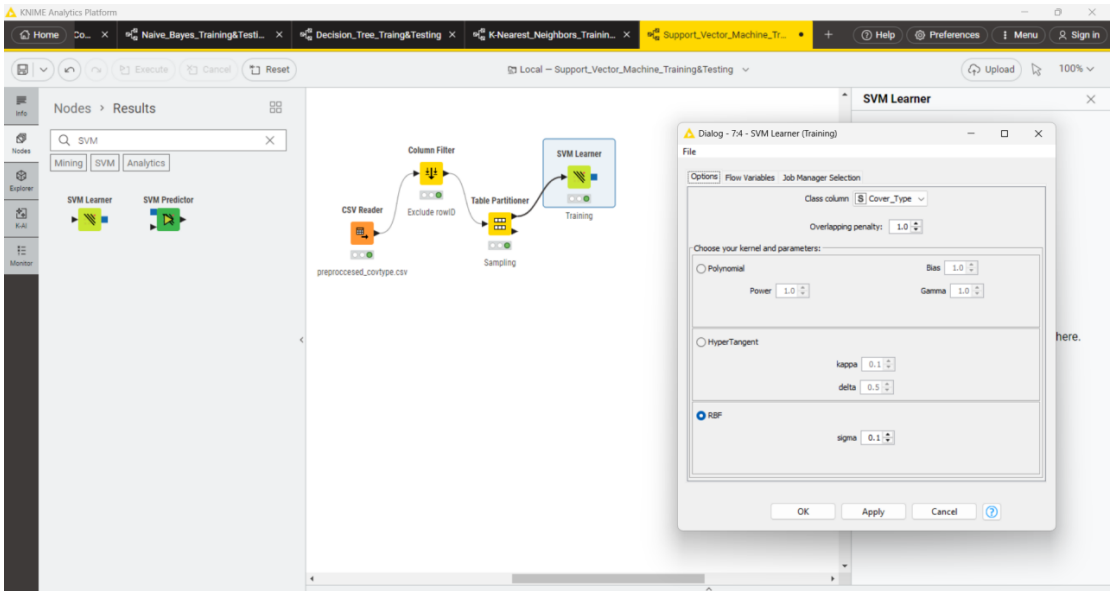


Figure 46: SVM Learner Node and Configuration

Since SVM Learner Node only does the training, there is no expected output from the node.

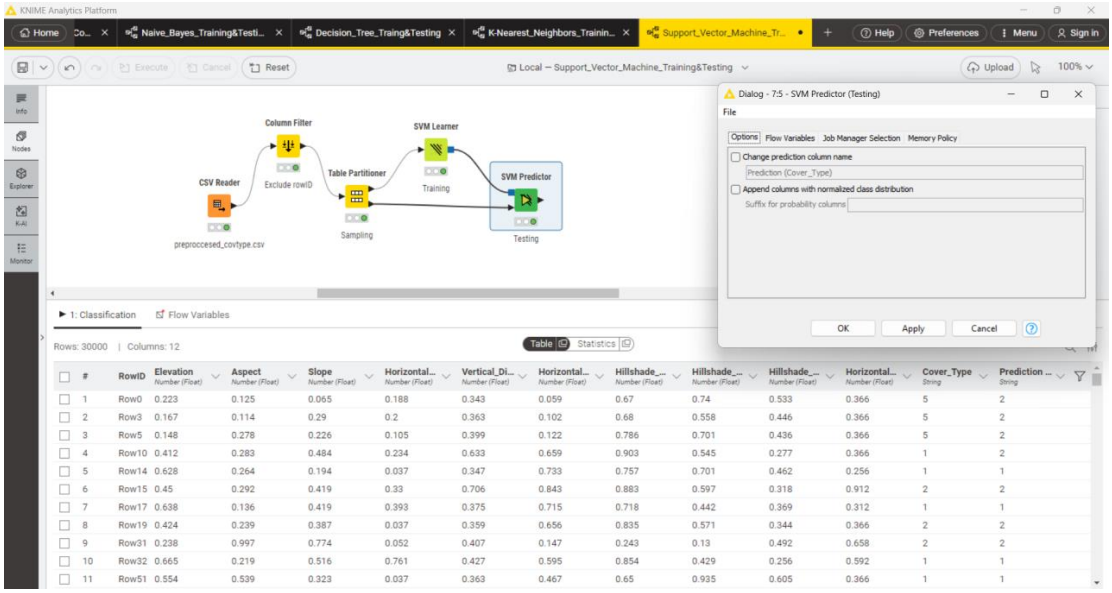


Figure 47: SVM Predictor Node, Configuration, and Output

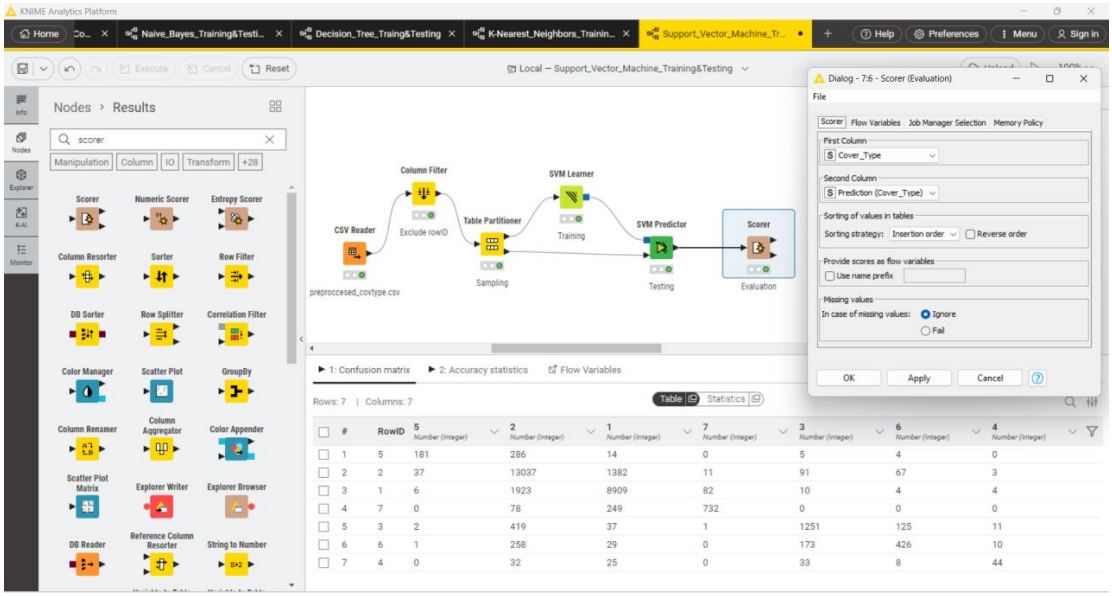


Figure 48: Scorer Node, Configuration, and Output - Confusion Matrix

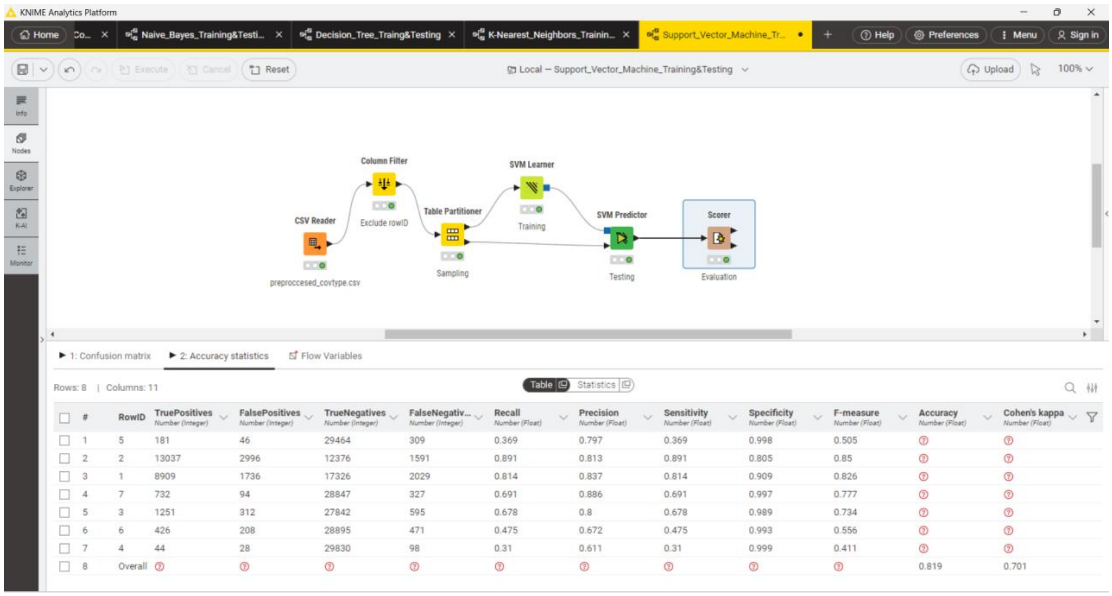


Figure 49: Scorer Node and Output - Accuracy Statistics

A **Support Vector Machine (SVM) classifier** was developed using the preprocessed dataset.

Prior to splitting the data, a **Column Filter node** was used to remove the *row ID* column, ensuring that only meaningful predictive features and the target variable were passed into the model.

The dataset was then split into 70% training and 30% testing using **stratified sampling** to maintain balanced representation across all cover types.

The **SVM Learner node** was configured with *Cover_Type* as the target attribute and the **RBF kernel**, which is well-suited for non-linear classification tasks and performs effectively with normalized numerical data.

The trained model was applied to the test partition using the **SVM Predictor**, and performance metrics such as accuracy, precision, recall, F-measure, and the confusion matrix were obtained using the **Scorer node**.

(e) Neural Network

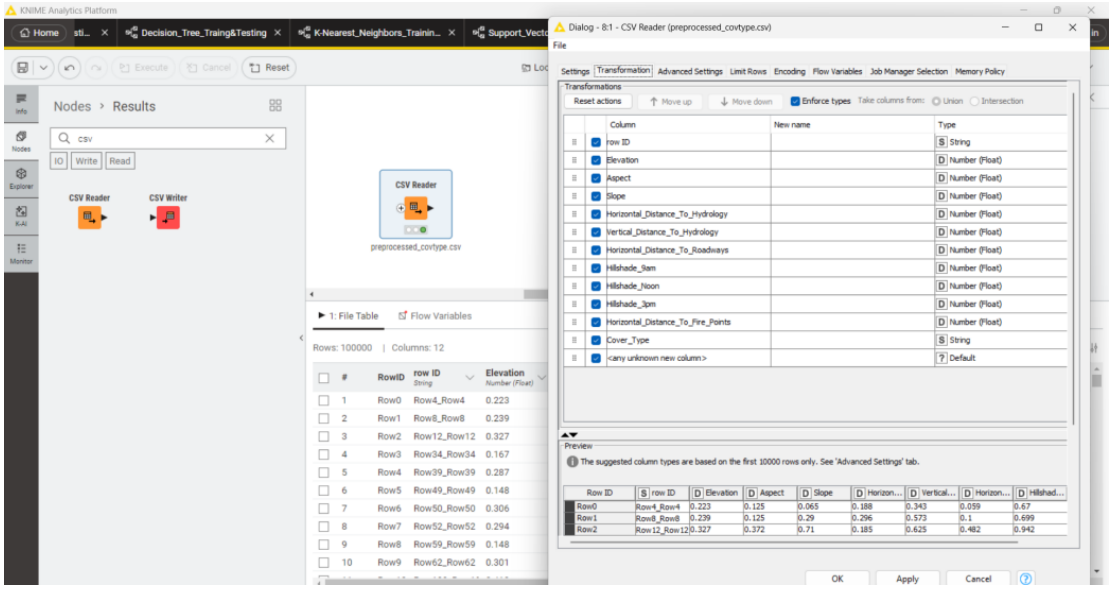


Figure 50: CSV Reader Node, Configuration, and Output Table

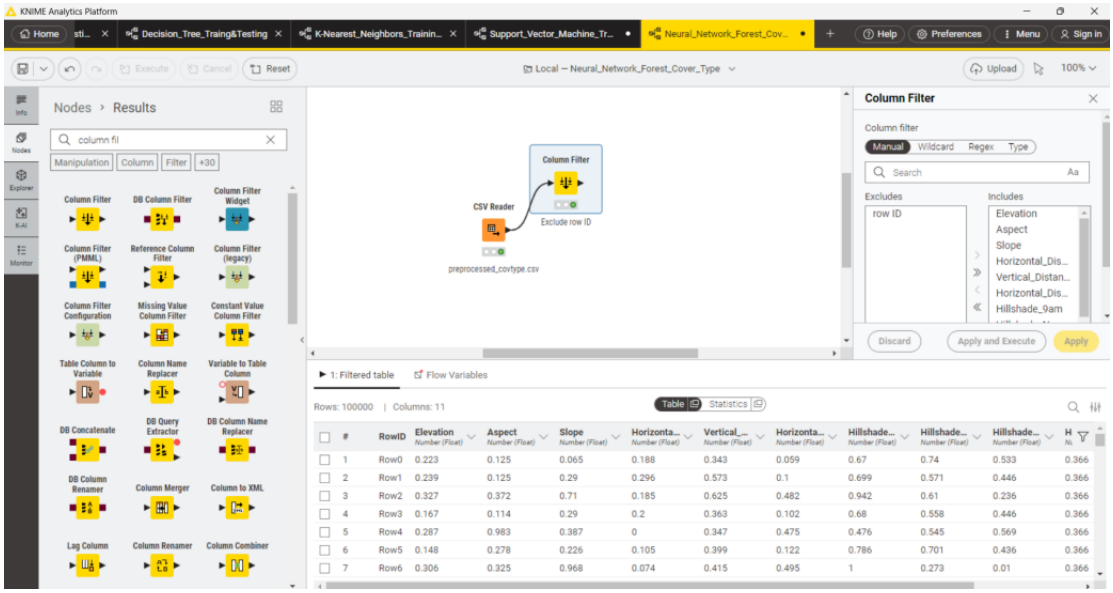


Figure 51: Column Filter Node, Configuration, and Output Table

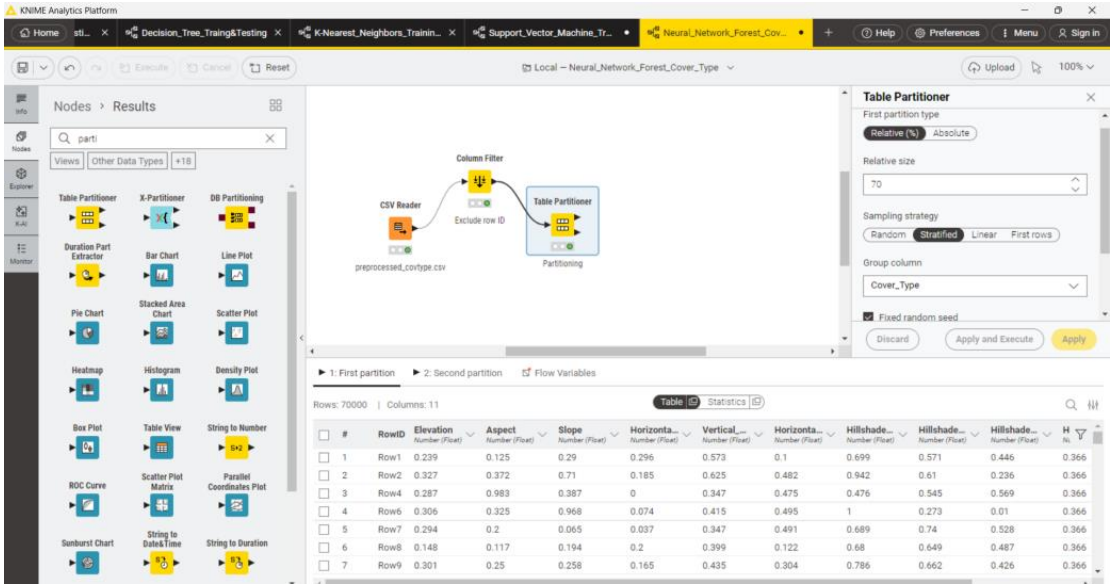


Figure 52: Table Partitioner Node, Configuration, and First Partition (70000)

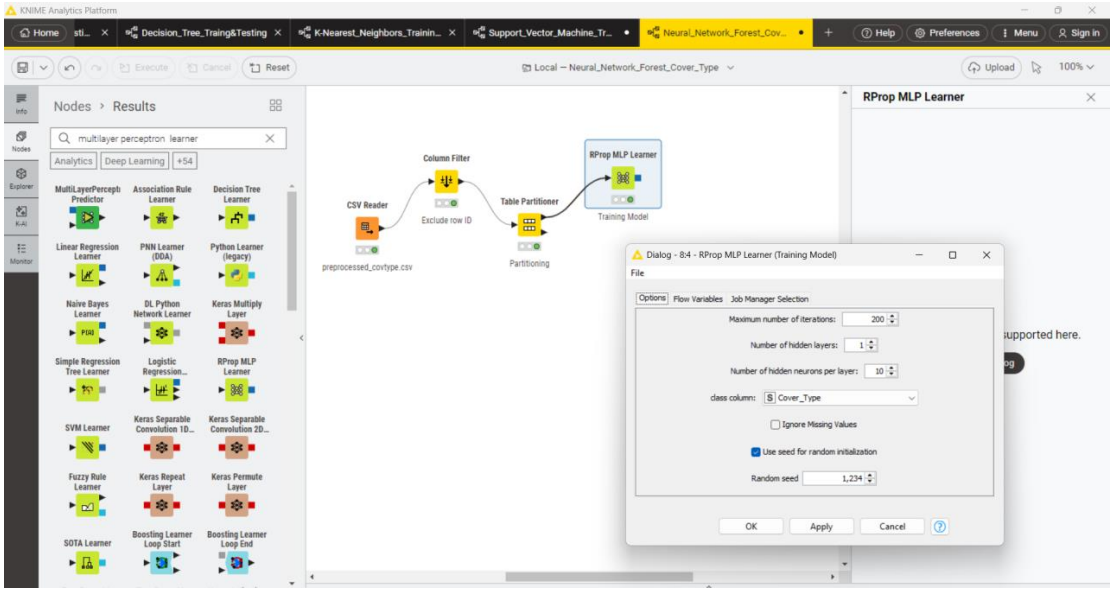


Figure 53: RProp MLP Learner Node and Configuration

Since RProp MLP Learner Node only does the training, no expected output.

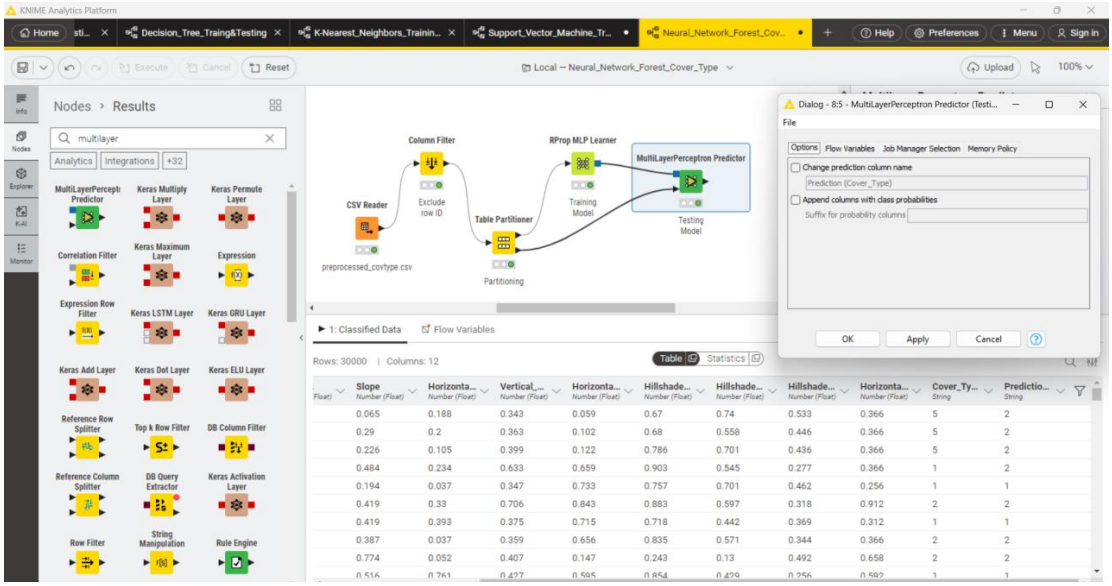


Figure 54: Multilayer Perceptron Learner Node, Configuration, and Output Table

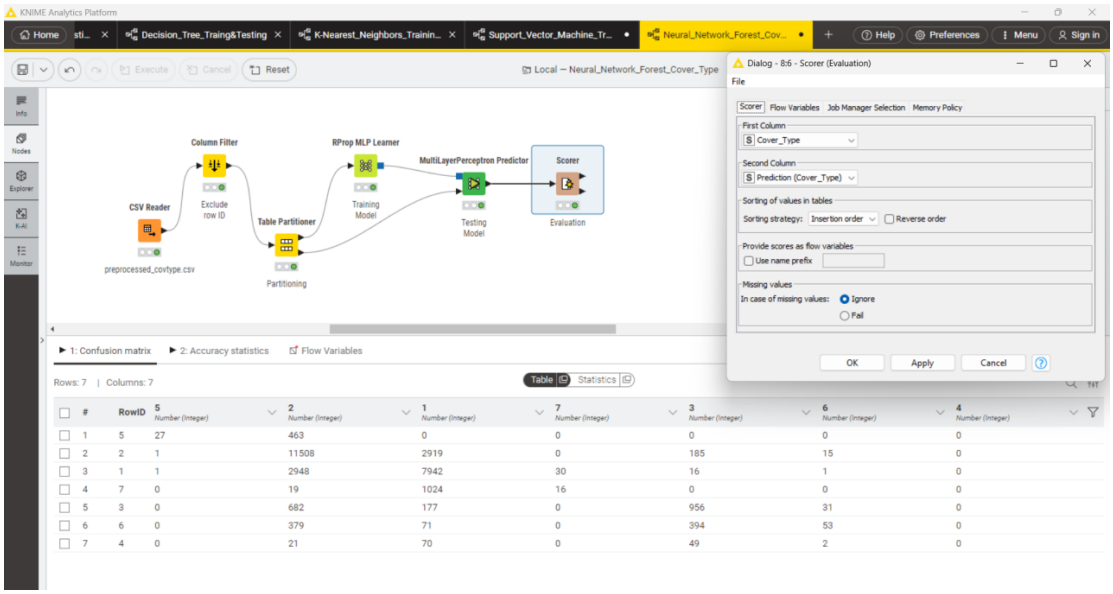


Figure 55: Scorer Node, Configuration, and Output Table - Confusion Matrix

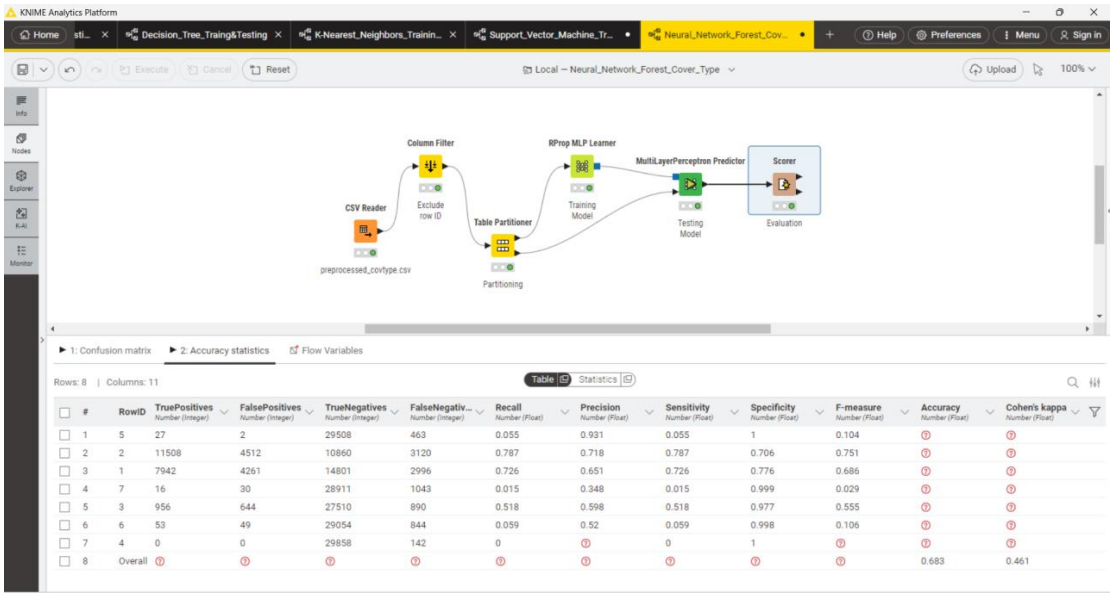


Figure 56: Scorer Node and Output Table - Accuracy Statistics

A **Neural Network classifier** was developed using the **RProp MLP Learner**, which implements a multi-layer perceptron trained with resilient back-propagation.

Before training, a **Column Filter node** was used to remove the non-predictive *row ID* attribute, ensuring that only meaningful numerical features and the target label were included.

The dataset was then divided into 70% training and 30% testing using stratified sampling on the *Cover_Type* variable to maintain balanced class representation. **The RProp MLP Learner node** was configured with one hidden layer containing ten neurons, providing a baseline architecture that is simple but effective and suitable for the normalized dataset. The trained neural network model was applied to the test set using the **Multilayer Perceptron Predictor node.**

Performance metrics such as accuracy, precision, recall, F-measure, and the confusion matrix were generated using the **Scorer node**.